



UNIVERSITE CATHOLIQUE DE LOUVAIN
ECOLE POLYTECHNIQUE DE LOUVAIN
COMPUTER ENGINEERING DEPARTMENT

MASTER THESIS

Graph and set variables implementation in OslaR

Supervisor:

Pierre SCHAUS

Readers:

Yves DEVILLE

Renaud HARTERT

Thesis submitted for the

degree of Master in

Computer Science Engineering

Author:

Andrew LAMBERT

June 1, 2014

Acknowledgements

I would like to thank my supervisor Pierre Schaus who proposed this thesis and allowed me to work on this subject. His implementation of the set variables also really helped me to understand how things should be done in *Oscar* and allowed me to start in an efficient way on the graph variables and their implementation.

I would like to thank Renaud Hartert who helped me throughout this master thesis. He helped me to solve many problems I faced during the realization of this work. We also had very interesting discussions about various topics which helped me greatly to acquire a better understanding of the concepts.

I would like to thank my brother who helped me to improve significantly the quality of this paper.

Finally, I would like to thank Salomé Mulders who helped and supported me throughout this year.

Contents

1	Introduction	4
2	Background	6
2.1	Constraint programming	6
2.2	Graph theory	7
2.3	OscAR	9
3	Graph and set variables in Constraint Programming	10
3.1	Set variable	10
3.2	Graph representation	14
3.3	Graph implementation	16
4	Constraints	20
4.1	Modify the graph variable implementation	20
4.2	Graph binary relations	22
4.2.1	SubGraph	23
4.2.2	Graph complement	25
4.2.3	Induced SubGraph	25
4.3	Connectedness Constraints	27
4.3.1	Undirected connected graph	27
4.3.2	Graph weakly connected	29
4.3.3	Graph strongly connected	30
4.4	No-Cycle Constraints	30
4.4.1	Directed acyclic graph	30
4.4.2	Graph bipartite	35
4.4.3	Tree	36
4.5	Path Constraints	37
4.5.1	Simple path	38
4.5.2	Shorter path	43
5	Applications and experiments	47
5.1	Travel Salesman Problem	47
5.1.1	Integer model	47
5.1.2	Graph model	49

5.1.3	Comparison	52
6	Conclusion and future work	57
A	TSP code examples	58
A.1	TSP instances and useful functions	58
A.2	Integer Model	58
A.3	Graph Model	59

Chapter 1

Introduction

The resolution of combinatorial problems occurs in many fields such as biology, network design and logistics. A significant part of these problems can be seen as graph problems. Communication networks, route planning, circuitry, and bioinformatics problems are indeed cited in [1] to be modeled as combinatorial graph problems. The focus in [1] was to model and analyze biochemical networks. Such networks can be modeled as graphs where the cells are the nodes and their interactions are the edges. Additional applications are presented in [2]: the author models a vehicle routing problem and a phylogenetics problem. Phylogenetics problems can indeed be modeled as graph problems where the nodes represent the individuals and the edges the relation between these individuals.

Various kinds of problems can thus be modeled as graph problems and an interesting approach to express such problems is to use the constraint programming paradigm. Not long ago, a new dedicated graph variable computation domain was proposed in [1] for the constraint programming paradigm. This graph variable allows to solve graph problems that can be translated to constrained subgraph finding problems. The idea was to use graph properties to model problems using graph variables and to solve them more efficiently. Recent works such as [3] have indeed shown that using propagators based on graph properties can improve search heuristics behaviors. Constraints dedicated to these graph variables and their associated pruning rules have been proposed in [4].

The goal of this paper is to implement set and graph variables in the constraint programming package of the *Oscar* solver [5], along with global constraints on these graph variables. All the source codes that were implemented and that are presented in this work are available in the project *Bitbucket* repository [6]. This repository is a fork of *Oscar*: we created a copy of the original *Oscar* repository to add the new graph variables and associated constraints.

Graph variables have already been developed and implemented in other solvers. G. Doms and Z. Zampelli for example have implemented set and graph variables in the *Gecode* solver. Their implementation of graph variables and some associated constraints as a contribution package to the generic constraint development environment *Gecode* are presented in [7] and in [4]. How-

ever, this contribution has not been maintained and was then removed from the *Gecode* solver.

Another example of solver in which graph variables have been implemented is the *Choco3* solver [8]. A description of the graph variables implemented in this solver is provided in [9]. This solver uses the same notion of graph variable but the notations are different. These notations call the set of required elements of the graph domain the *kernel* and the set of possible values the *envelope*. The difference between this work and the one made in the *Choco3* solver lies in fact in the implementation of graph variables. The *Choco3* implementation defines a graph variable as a set of successors of each node and a set of predecessors of each node. Edges are not directly used. The notion of activation is then used to know if a node belongs to the graph domain. If a node does no longer belong to the domain, it will be deactivated. The graph variable implementation that will be presented in this work will represent a graph variable as a set variable for the nodes and a set variable for the edges. However, both solvers use a Sparse-Set representations for set domains implementations. Sparse-Set for domain implementation are presented in [10].

This work will be organized as follows. The next chapter will provide an overview of the constraint programming paradigm, some definitions and properties from graph theory, and a description of the Operational Research solver *OscAR*. The third chapter will explain how set and graph variables computation domains are modeled and implemented in *OscAR*. The fourth chapter will detail global constraints over graph variables. The fifth chapter will present two modelizations of the travel salesman problem and compare them with each other. Finally, in the last chapter, we will conclude this work by summarizing the main results and discussing future works which could be achieved on the subject.

Chapter 2

Background

This chapter will explain basic notions (along with some more specific ones) that will be used throughout this work. We will start first by explaining the basics of the constraint programming paradigm. Then we will explain graph theory principles along with some definitions. Finally we will provide a short description of *OscAR* [5].

2.1 Constraint programming

Two good references to learn in detail the principles and mathematical models beyond the constraint programming paradigm are [11] and [4].

Here, we will just explain basic notions of constraint programming (CP) to have a better understanding of the concepts used throughout this paper.

Constraint programming is a programming paradigm used to solve Constraint Satisfaction Problems (CSP) and Constraint Optimization Problems (COP). This paradigm states the relations between the variables in the form of constraints. A solution of a CSP will be an assignment of all variables to a value such that all constraints are satisfied. A solution to a COP will also be a solution to the relaxed CSP but will also maximize/minimize an objective function.

To solve a given problem with CP, we first have to model it; this means that we have to choose variables and their domain accordingly. Then we will have to post constraints over these variables to link the variables and model the problem. The third step is the search. Considering that the problem is a constraint satisfaction problem, the search will look in the search space an assignment of variable such that it respects all constraints. Such an assignment will be the solution of the CSP.

The computation will be an iteration of two main steps :

Propagation the goal of the propagation is to reduce the domain of the variables by detecting impossible values of variables. By doing this, we find an equivalent problem to the original one with smaller domains of variables.

Branching the goal of the branching is to make a choice during the search by decomposing the problem into sub-problems and solve them independently. This choice is non-deterministic and a heuristic is often used to guide the search leading to a better pruning of the search space

In this work, we will implement and use global constraints. Global constraints are generally non-binary constraints which provide a better filtering than its decomposition into simpler constraints. Specialized global constraints can indeed improve the quality of the pruning (handling multiple simpler constraints at the same time can prune variables domains more efficiently) and the computation of the pruning (dedicated filtering can achieve time or space complexity more efficiently), along with allowing to model more easily and in a more readable way.

2.2 Graph theory

As in the previous section, we will just provide here a basic explanation of the concepts of graph theory and the notions that will be used further in this paper. To have a more complete view, you can take a look at [12], [13] and [4].

In mathematics and computer science, graph theory is defined as the study of structures used to model relations between two kinds of objects. More precisely, we will define here a graph $G = (N, E)$ as a set of nodes N and a set of edges $E : (N \times N)$.

A directed graph is a graph where all edges have direction, with each edge $e : (u, v)$ having a source u and a destination v . An example of directed graph is shown in figure 2.1. On this figure, we can see 3 nodes and 4 edges. There is also another sort of graph. The undirected graph is a graph where edges have no direction $(u, v) = (v, u)$.

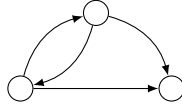


Figure 2.1: Example of a directed graph

The neighbors of a node u are the set of all nodes $v \in N$ such that there is an edge between u and v in E . For a directed graph, we can split the neighbors of a node into two categories. The in-neighbors of u are the nodes such that there is an incoming edge to $u : v|(v, u) \in E$. Symmetrically, the out-neighbors are the nodes such that there is an outgoing edge from $u : v|(u, v) \in E$.

A graph $G_1 = (V_1, E_1)$ is a subgraph of $G_2 = (V_2, E_2)$ if and only if $V_1 \subseteq V_2$ and $E_1 \subseteq E_2$. This notion is different from the induced subgraph. G_1 is an induced subgraph of G_2 if and only if any edge (u, v) in G_2 incident to nodes of G_1 must also be present in G_1 .

A complete graph of order n is a graph with n nodes such that there is an edge connecting every possible pair of nodes.

A graph is said to be the complement of another graph if they have the same number of nodes n and if each edge of the complete graph with n nodes belongs to one and only one of the two graphs.

A graph is called bipartite if its nodes N can be split into two sets N_1, N_2 such that there is no edge in $N_1 \times N_1$ or $N_2 \times N_2$.

A path in a graph is a sequence of non-repeated nodes and edges starting from a node and ending at another node. A circuit is a path starting from a node and ending at the same node. A directed acyclic graph (DAG) is a directed graph with no circuit. An example of a DAG is shown in figure 2.2a. In this graph, there is indeed no circuit.

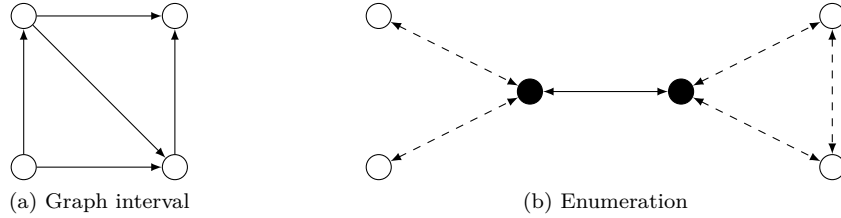


Figure 2.2: Examples of graph properties

A graph is said to be connected if there exists a path between each pair of nodes of the graph. A cutnode of a graph is a node whose removal would disconnect the graph. A bridge of a graph is an edge whose removal would disconnect the graph. An example of a cutnode and of a bridge is shown on figure 2.2b. The cutnodes are the nodes filled in black and the bridge is the straight black line. We can indeed see that if we remove one of the cutnodes and/or the bridge, we disconnect the graph.

The transitive closure of a graph $G = (N, E)$ is the graph $G^+ = (N, E^+)$ where E^+ is the set of edges such that $\{(u, v) \in E^+ \mid \text{exists path from } u \text{ to } v\}$. We have that $G \subseteq G^+$ as G^+ has the same edges than G and it is also composed of paths between reachable nodes. A representation of a graph G and its transitive closure G^+ is shown on figure 2.3. The dashed edges are the edges which only belong to G^+ .



Figure 2.3: Example of a graph G and its transitive closure G^+

2.3 OsaR

OsaR [5] is a Scala toolkit for solving Operations Research problems. The techniques currently available in *OsaR* are split into the following packages:

- cp: a package to solve Constraint Programming problems
- cbls: a package to solve problems with Constraint-Based Local Search
- linprog: a package to write Linear Programming/Mixed Integer Programming models
- dfo: a package to solve Derivative Free Optimization problems
- des: a package to model Discrete Event Simulation problems
- visu: a package to build quickly visualizations of optimization problem

This project is kindly supported by some companies/institutions : *UCLouvain* and the *BeCool* research lab, *CETIC* which develops and maintains the *CBLIS* package and its satellite engines, *n-Side* which uses *OsaR* and allocates resources to improve it, *YourKit* is kindly supporting this open source project with its full-featured Java Profiler.

In this work, we will focus on the constraint programming package. This package was initially developed by P. Schaus and has been mainly improved by P. Schaus and R. Hartert.

For additional information, you can visit the *Bitbucket* repository [5]. There is also a nice online tutorial which is currently developed by P. Schaus : [14] in which we can learn *CP* and *OsaR*. It allows to get started really fast with *OsaR* and to easily understand the concepts of constraint programming thanks to useful examples.

Chapter 3

Graph and set variables in Constraint Programming

In this section, we will first define the set variable in constraint programming. Then, we will explain how it is modeled and implemented in *Oscar*. After that, we will define graph variables in CP. Using the previously defined set variables, we will be able to model graph variables in *Oscar* in an efficient and compact way. We will finish this chapter by explaining how we have implemented these graphs.

3.1 Set variable

As we can remember from section 2.1, the constraint programming paradigm consists in posting constraints over variables domains and then reduce domains to find a solution that satisfies these constraints. For set variables, the discrete domain will be, as its name suggests it, sets. The domain abstraction used for the sets will be the set interval. This domain abstraction was introduced in 1997 by C. Gervet in [15].

Definition 3.1 (Set interval) *Set intervals are defined by a greatest lower bound (glb) and a lowest upper bound (lub), the glb being the set of elements that must belong to the set variable and the lub being the set of elements that may belong to the set variable.*

Set intervals are defined more formally in [4]. In a set interval, each element belonging to the glb is required to form it and each element which is in the lub but not in glb may be included. A standard notation for these set variables is the following : having a set variable S , the set of the elements of the domain belonging to the glb can be written as $\underline{D}(S)$ and the set of the elements belonging to the lub $\overline{D}(S)$.

Additionally, the following property holds:

Property 3.1 *For all set variables S , the greatest lower bound $\underline{D}(S)$ is included in the lowest upper bound $\overline{D}(S) : \underline{D}(S) \subseteq \overline{D}(S)$.*

This definition of set variables as set intervals is very clear when we use an example. On figure 3.1, we can see that there is only one element in the *glb* $\underline{D}(S) = \{1\}$, and that there are three elements in the *lub*, $\overline{D}(S) = \{1, 2, 3\}$. The set interval on figure 3.1 represents all possible intervals composed of at least the elements 1 and that may contain the elements 2 and/or 3. This set interval is equivalent to the following set : $\{\{1\}, \{1, 2\}, \{1, 3\}, \{1, 2, 3\}\}$.

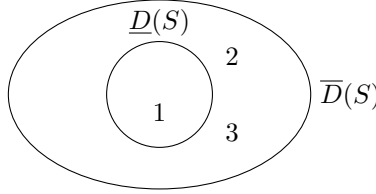


Figure 3.1: Example of a set interval

We can see that this compact way to encode set variables can be very efficient if we want to have set variables containing lots of variables.

But if we want to implement these set variables in practice, we would have to keep a lower bound and an upper bound for our set variables.

We will now explain, without entering into too much details, how these set variables have been implemented in *Oscar* by *P. Schaus*. Readers interested into this can take a look at the source code in the package `oscar.cp.core`, and more precisely in the class `CPSetVar.scala` of *Oscar* [5]. Additionally, a discussion about the usage of sparse set for integer domain can be found in [10]. These sparse sets were implemented and used in *Oscar*.

Let's assume that we want a set variable containing values between a value *min* and a value *max*. To have it, we will need a domain $[min, max]$.

```
private val dom = new SetDomain(s, min, max);
```

We will also need some reversible pointers to take the constraints over the set variables into account and to propagate domain modifications but we will focus here on domains. Before doing that, we need to define and explain what a `SetDomain` is.

In fact, the `SetDomain()` is simply a `ReversibleSparseSubset()` with additional getters and setters. We will take a look into this `ReversibleSparseSubset()` to understand how set variables are done in practice. The definition of it is the following one :

```
ReversibleSparseSubset(store: ReversibleContext, val min: Int, val max:
Int)
```

The first argument is a `ReversibleContext`; it represents a reversible state (a state which is able to restore the previous visited states during the search). This reversible context allows the set to update its values during the search of the solution when solving a Constraint Satisfaction Problem. When we go deeper in a search tree, we indeed need a way to come back to a previous state in case of Failure in order to continue the search. This context will store the previously visited states and will allow backtracking. The two other arguments are simply the minimal and the maximal values of the set. Now, we need to define variables that will keep track of the greatest lower bound *glb* and of the lowest upper bound *lub* as defined in the set interval model. We will define two sizes, the first one representing the subset (the number of elements in the *glb*) and the second one the super-set (the number of elements in the *lub*). Initially, the subset is empty and the super-set contains every elements. Once again, we will use two `ReversibleInt` to store the sizes to allow backtrack into a previous state when we search for a solution.

```
val size1 = new ReversibleInt(store, 0)
val size2 = new ReversibleInt(store, max - min + 1)
```

In practice, we will also need two arrays to keep indexes and values in the set. But to simplify, we will only consider the array holding values in this paper.

Now that we have properly understood how a domain works, we will focus on the operations on this domain and their implications on the set variable. To be useful, two operations must be defined on this set variable:

requires it adds an element to the required values (the element will now belong to the *glb*)

excludes it removes an element from the possible values (the element will now be removed from the *lub*)

In order to have a clear and understandable explanation, we will use an example and a visual representation of the variable using two sizes `size1`, `size2` as defined previously. Let's create a set variable having initially an empty greatest lower bound *glb* and a lowest upper bound *lub* containing the following values : {2, 3, 4, 5, 6}. At the beginning, we can represent this set variable as an array with the variable *size1* being equal to 0 (no elements are present in the *glb*) and the variable *size2* being equal to 5 (all the elements are present in the *lub*). This is illustrated on figure 3.2a.

Now, we will add an element to the required set. For example, we will add the value 4 to the required set : `required(4)`. To do this, we switch the value 4 and the first value in the array and we increase the size of `size1`. By doing this, we now have the element 4 in first position in the array and the variable *size1* is equal to 1, as we can see on figure 3.2b. If we remember that `size1` represents the number of elements in the *glb*, we can see that the value 4 is included in the required set. If we want to take an instance of the set, this value must therefore

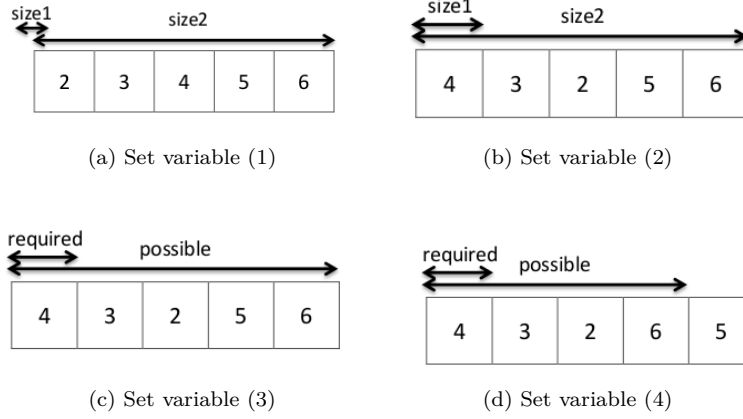


Figure 3.2: Example of a set variable

be part of it. Another way to see this is represented on figure 3.2c. Now, we will remove an element from the possible set, for example we will exclude the value 5 from the set variable: `exclude(5)`. For an exclusion, we will use the same kind of pattern as for an inclusion; we will switch the value 5 and the last element of the array (more precisely, we will switch the element to be excluded with the last possible element of the set) and we will reduce the value of the variable `size2` by 1. The result can be seen on figure 3.2d. The value 5 is no longer possible in the set.

In general, we can represent set variables as on figure 3.3. Some other cases that the presented ones can also happen. Trying to exclude a required value for example should not be possible, trying to require a not possible (or forbidden) value should also not be possible, but they are implementation details and don't require further explanations. The goal here is to understand the set variables and their implementation in practice and not to explain every detail of their functioning and implementation.

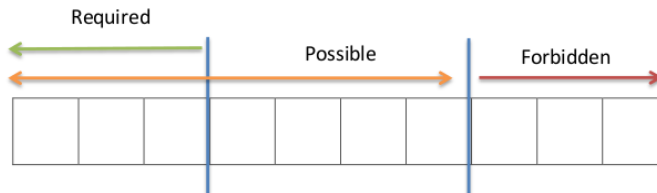


Figure 3.3: Representation of a set variable

But the above set variable implementation is not very useful alone. In order to use it in a constraint programming solver, we have to define constraints,

propagators and filtering rules over these set variables.

A theoretical analysis of basic constraints and filtering rules over set variables may be found in [4] and [15]. But we will not describe it in this paper.

There is an implementation of some constraints and their filtering rules which was made by *P. Schaus* in *Oscar*. The complete implementation details will not be presented in this master thesis but readers interested into this can take a look at this work, and more precisely at the class `CPSetVar.scala` from the *CP* package of *Oscar* and at all the constraints defined over set variables in the *CP* package. The source code is available on the *Bitbucket* repository [5].

3.2 Graph representation

Such as for set variables, we want to find a domain abstraction for graphs. In *CP*, we post constraints over variable domains. A possible domain for graphs would be the set of all possible graphs. But such enumeration would require a lot of space and is not very convenient to work with. G. Doms, Y. Deville and P. Dupont introduced in [1] an new and efficient domain abstraction for graphs: $CP(\text{Graph})$. This notion was further developed in [4]. We now detail this domain abstraction that is used to model our graph variables. We will first define a graph domain as a graph interval:

Definition 3.2 (Graph interval) *Let G_1, G_2 two graphs such that $G_1 \subseteq G_2$, then $[G_1, G_2]$ is the set of all graphs G such that $G_1 \subseteq G \subseteq G_2$. We call it a graph interval.*

We can see similarities between graph intervals and set intervals. In both cases, we have a lower bound containing all mandatory elements and an upper bound containing all possible elements of the interval. In the rest of this work, we will also call the mandatory elements the required elements to keep the same notations as the ones used for the set variables and the `require/exclude` operations. We can use the notations : $G_1 = \underline{D}(G)$ and $G_2 = \overline{D}(G)$. Our graph interval will be $G := [\underline{D}(G), \overline{D}(G)]$.

We can see an example of graph interval on figure 3.4a. In this example, we have represented the required nodes and edges in solid and the possible ones with dashed lines and empty nodes. The second part of the figure 3.4b is an enumeration of all the possible graphs. Both are the representation of the same graph but we can already see the power of abstraction: a graph interval composed of only two set variables is equivalent to an enumeration of graphs.

We will now use this abstraction - *graph intervals as domains for graph variables* - and the set variables as defined in the previous section to model our graph variables.

Let us first remember what a graph is. As described in section 2.2, a graph is composed of nodes and edges and models the relation between them. In fact, a graph may be represented as set of nodes and a set of edges. Let us define it more formally.

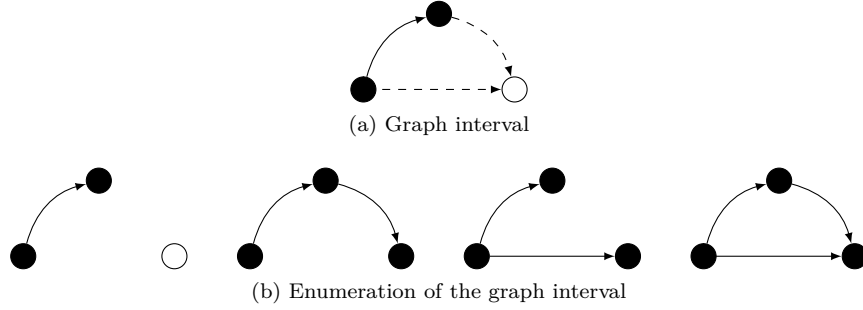


Figure 3.4: Example of a graph interval

Definition 3.3 (Graph) *Let SN be the set of nodes, SE be the set of edges : $SE \subseteq SN \times SN$. We can define a graph G as $G := (SN, SA)$.*

We have now defined a graph. But, in *CP*, we want to have variables over domains and we want to define a graph variable. To be able to do this, we simply have to use the set variables as they were defined in the previous section. We can then define a graph variable as a set variable of nodes and a set variable of edges. For nodes, it is obvious because we go from a set of integer to a set variable of integer. For edges, it seems more tricky at first sight but in fact this is not really that hard. An edge is defined as a pair of nodes: $e : (n_1, n_2)$. But, if we give a integer value to each edge (with an integer representing a specific edge), our edges are simply a set of integer values. Of course, we have to remember which integer value corresponds to which edge and be sure that only one value corresponds to one edge and conversely but this is an implementation detail at this stage. We will explain in the next section how we did it in practice.

Up to now we have defined directed graphs. But what if we want to work with undirected graphs? In this case, we can simply model an undirected graph as a directed graph: each undirected edge is split into two directed edges which are in opposite direction (e.g. undirected edge $(1, 2)$ would be split in our directed graphs model into edges $(1, 2)$ and $(2, 1)$). Conversely, if we want an undirected graph from a directed one, we can either forget the orientation of the edges, or only merge two opposite edges. But we will not go further on this in this paper. We will indeed work with directed graph here because directed graphs are the more general model and given it is easy to go from one representation to the other. It does not bring us anything to use two different models.

3.3 Graph implementation

In this section, we will explain what we have done to implement the graph representation explained above in *Oscar*.

First of all, our graph variable representation: `CPGraphVar` will take three arguments by default: the `CPStore`, the number of nodes and a list of tuple which represents the edges.

```
class CPGraphVar(val s: CPStore, nNodes: Int, inputEdges:
    List[(Int,Int)], val name: String = "") extends CPVar { ... }
```

The `CPStore` is the object representing the *CP* solver; a `CPGraphVar` object will indeed be used in a *CSP* and therefore we need a connection to the solver store. In order to compute the bounds of our graph interval, we decide that the number of nodes and the edges would form the upper bound, namely the set of all possible values in the graph. The lower bound will be the empty graph. For the nodes, we only need a value which is the number of nodes. A value which represents the number of nodes is indeed sufficient to create a list of integers which will be the set of nodes, and giving a single integer value is furthermore more convenient for a user. The node numeration will go from 0 to $n - 1$, with n standing for the number of nodes. Additionally, we will also have to provide a list of edges which is represented by a list of tuple of integers. This is the easiest way to provide all the required information.

However, it is not very convenient for a user to put all edges in parameter if he wants to use a complete graph. To solve this, we use the companion object of `CPGraphVar`. We allow two possible constructors to be used to create a graph variable:

```
def apply(nNodes: Int, inputEdges: List[(Int,Int)])(implicit store:
    CPStore) = { ... }
def apply(nNodes: Int)(implicit store: CPStore) = { ... }
```

This allows the user to enter all the edges of the graph or to get a complete graph by just providing the number of nodes (the set of the edges corresponding to the complete graph is built automatically).

We can now take a deeper look into the body of the class. The class is composed of two `CPSetVar` objects, a structure which is the internal representation of the graph, accessors (also called getters) and modifiers (also called setters). We will now detail each part of this class:

Two CPSetVar These two `CPSetVar` are really the core of the `CPGraphVar`. As a matter of fact, using the set variables for our graph variable will make the `CPGraphVar` implementation easy. We need one set variable to model nodes and one to model edges.

```
private val N = new CPSetVar(store,0,nNodes-1)
private val E = new CPSetVar(store,0,inputEdges.length-1)
```

The access modifiers are private because we do not want to allow a third party to modify those core items directly. To do so, he will have to use the getters and setters.

Internal representation As explained earlier, we need an internal representation to match the integer set standing for the edges and the actual edges. To do this, we will first define two classes that represent a specific edge and a specific node with useful elements:

```
case class Edge(index : Int, src: Int, dest : Int)
case class Node(index : Int, inEdges : List[Int], outEdges :
  List[Int])
```

By using such objects, it will be easy to access source/destination node from an edge and the incoming/outgoing edge(s) from a node. We will use these objects to create the internal structure which links the `CPSetVar` and the actual graph:

```
private val edges : IndexedSeq[Edge] = Array.tabulate(nEdges)(i
  => new Edge(i,inputEdges(i)._1,inputEdges(i)._2 ) )
private val nodes : IndexedSeq[Node] = ...
```

We use indexed sequences to set our sequences immutable and to prevent this structure to change. Actually, only the two `CPSetVar` will be modified to update the graph interval to its current state when we solve a *CSP*. Also, the fraction of code presented here is also nor complete nor perfectly correct but what is interesting to explain here is the idea beyond the implementation. To be complete, we also have to check that the `inputEdges` from the user are in the range of the set of nodes. Readers interested in this can find the complete implementation in the *Oscar* [5] repository, and more precisely in the class *CPGraphVar*.

Accessors As we do not want that a user could access our implementation directly, we created some accessors to allow a user to get all the information on the nodes and the edges that he might need. We will explain two accessors here, others having been implemented following the same pattern.

First, we would like to have a method giving is the possible nodes of the graph interval. If we remember how we implemented a graph interval, the list of possible nodes is simply all the possible nodes from the `CPSetVar` representing the graph nodes. A `CPSetVar` object has a method `possibleSet` which gets all possible nodes in a single call. We can use this method directly to obtain our set of possible nodes:

```
def possibleNodes() : List[Int] = N.possibleSet.toList
```

For edges, this is a little bit more complicated. Let us assume that we want to retrieve all the required edges (in fact, required edges in the graph interval are edges belonging to the lower bound of the interval) coming in the node with the index `nodeId`. These *in* edges are edges such that their destination is this node. We can retrieve the list of all the incoming edges from the node `nodeId` by using the structure: in the `nodes` indexed sequence, we can simply take the node with the index `nodeId` and get all `inEdges`. After that, we have to filter this list with the corresponding `CPSetVar` to get only the required nodes:

```
def requiredInEdges(nodeId: Int) : List[Int] =
    nodes(nodeId).inEdges.filter( E isRequired _ )
```

To access all the elements of the graph, we defined many accessors but we will not give the implementation details, the idea being basically the same behind all accessors. With these two examples, we have seen how our internal structure linking the edges and the integer value which represents it in the `CPSetVar` can be used with the two `CPSetVar` to get all the information that we need about the graph variable.

Modifiers A structure is useful to represent and access the graph. But, we also need to implement modifiers in order to have a graph variable which can be used in *CP* and modified by propagators.

The four accessors for our graph variable are: add a node, remove a node, add an edge, remove an edge. But what exactly does it mean to add a node? If we remember our model and the `CPSetVar`, adding an element to a set variable is in fact set this mandatory, in other words add it to the required set. When we will take a graph from the graph interval, the required element must be taken. On the other hand, removing an element means that it will be forbidden and we will not be able to use it anymore. In the rest of this work, when we will talk about adding a node, it will be adding it to the required set of nodes of the set variable and the same for adding an edge to the required set of edges.

In this paper, we will only explain in details one accessor: the remove-node method. The first two accessors are indeed pretty self-explanatory. The method `addNodeToGraph()` simply uses the `requires()` method of the `CPSetVar` which handle the nodes and the method `removeEdgeFromGraph()` calls the `excludes()` method of the `CPSetVar` handling the edges. The two remaining accessors `removeNodeFromGraph()` and `addEdgeToGraph()` follow the same pattern which is detailed beneath.

We might wonder why we called these methods like this. For example, why `addNodeToGraph()` and not simply `addNode()`. The answer to this question will be presented in details in the next chapter when we will need to modify our `CPGraphVar` to deal with constraints. In short, we prefer to keep shorter method names for modifiers dealing with constraints and

propagation. When we use the graph variable in a *CSP*, we only need to modify the graph using a short method name; for example `addNode` to add a node to the graph:

```
add( graph.addNode(n) )
```

Let's end this section by explaining how we implemented the accessor `removeNodeFromGraph()`. As we will see beneath, we have to think about cases that might happen when we remove a node from a graph interval.

First, we try to exclude the node from the `CPVarSet N` holding the nodes. In case of `Failure` (a `Failure` could happen if the node is already required and then we are not allowed to remove it), we have to set the result of the call to the `removeNodeFromGraph()` method to `Failure`. The method will return a `Failure` if the removal does not succeed. If the call to the `excludes` method is correctly executed, then we have to remove all the edges linked to this node. Edges are indeed defined as a source and a destination and if one of them no longer exists, then the edge no longer exists either. Once again, the exclusion of the edges linked to the node could not be correctly executed and lead to a `Failure`. In this case, the output of the method should also be a `Failure`. If the method runs without errors, then it will return the outcome `Suspend` at the end to notify that everything was correctly executed.

An example of such a code is presented below:

```
def removeNodeFromGraph(nodeId: Int) : CPOutcome = {
  if ( N.excludes(nodeId) == Failure ) Failure
  else {
    for (e <- possibleEdges(nodeId))
      if ( E.excludes(e) == Failure) Failure
    Suspend
  }
}
```

Now, we have a efficient representation of the graph variable interval. The next chapter will talk about constraints and will start by the changes that have to be made to the `CPGraphVar` to be able to use it in a constraint satisfaction problem. We indeed need to handle search, propagation and posting graph modifications as constraints in order to solve Constraint Satisfaction Problems.

Chapter 4

Constraints

In this chapter, we will talk about constraints that can be used on our previously defined graph variables. We have implemented several constraints that enforce graph properties. Additional constraints on graph variables are presented in [4] and in [16], but they were not implemented here.

First, we will need to modify our class `CPGraphVar` in order to be able to call the `propagate` method of constraints. When there is a change in the domain of a variable (nodes or edges), we indeed have to recall the `propagate` method of each constraint posted on the graph variable to prune its domain. The constraints will then be able to remove inconsistent values from the domain (values that no longer respect the constraints) or to detect if the *CSP* becomes inconsistent. We will describe these modifications to the `CPGraphVar` class in the first part of this chapter.

After that, we will look at various constraints and how they were implemented. Some basic constraints will just be overlooked and we will explain in details more interesting ones such as the directed acyclic graph and the simple path constraints. This work will mainly explain the concept and show some parts of code. Readers willing more details can look in the source code in the package *oscar-cp*.

4.1 Modify the graph variable implementation

First, we want to register to domain modifications and call the `propagate` method of associated constraints. In fact, this operation is pretty easy when we use `CPSetVar` objects. Given that our graph representation uses two `CPSetVar`, registration to graph modifications will indeed simply call the method used to register to modifications of the corresponding `CPSetVar`, nodes or edges.

Let's remember that nodes are held in the `CPSetVar` denoted `N` and edges in the `CPSetVar` denoted `E`. Then, the registration to domain modifications is simply:

```
def callPropagateWhenDomainChanges(c: Constraint, trackDelta: Boolean =
  false) {
  N.callPropagateWhenDomainChanges(c, trackDelta)
  E.callPropagateWhenDomainChanges(c, trackDelta)
}
```

In the rest of this paper, when we will implement a constraint on a graph variable `g` and when we will register to domain modifications, we will simply call:

```
g.callPropagateWhenDomainChanges(this, false)
```

The other modifications to make to the `CPGraphVar` are the implementation of new setters. To be able to be used in a *CSP*, what we need is in fact a method to be called on a graph that adds/removes a node/edge and launches the *propagate* method of the associated constraints. As defined in section 3.3, the problem with setters is that they do not call the propagate method of a constraint when they modify the domain.

In order to solve this problem and to be consistent with the guidelines taken when implementing *Oscar*, we created the file `GraphConstraints`. This file will contain four classes, each one extending the class `Constraint` and bringing one modification to the graph variable: add/remove a node/edge. We will explain in details one of them, others having been made following the same pattern. We will explain here how we added a node in a graph variable.

We created the class `RequiresNode` and simply override the `setup()` method of a `Constraint`. The body of this method will call the method to add a node to the graph, modifying the graph as described in section 3.3. The implementation is the following:

```
class RequiresNode(val G: CPGraphVar, n: Int) extends Constraint(G.s,
  "Node required") {
  override def setup(l: CPPropagStrength): CPOutcome =
    G.addNodeToGraph(n)
}
```

When using this `RequiresNode` class, we are able to add a node to the graph as a new constraint (the class indeed extends the class `Constraint`). This may sound weird at first sight. In fact, it is very convenient because it allows us to launch the propagation of a constraint which is associated to this one and to branch on different alternatives during the search. The new method for adding a node in the class `CPGraphVar` will be pretty simple:

```
def addNode(n: Int) = new RequiresNode(this, n)
```

Given the fact that the method to add a node is a constraint - this sounds obvious if we think about constraint programming; we add a new constraint modifying the domain - we can add a node n in a *CSP* by posting an 'addNode' constraint on a graph variable g :

```
cp.post(g addNode n)
```

If there were constraints associated to the nodes of the graph g , then adding this node n will call the `propagate` of this constraint.

As we used the same pattern for the remaining modifiers (`removeNode()`, `addEdge()` and `removeEdge()`), and it doesn't bring anything more, we will not detail them here. Now, everything is ready in the `CPGraphVar` class to support constraints over graph variables.

In the next sections, we will explain the constraints we implemented and how we did it. The first section will explain in details how to implement a constraint in *Oscar* as it is the first constraint on a graph variable. After that, we will take a look at all implemented constraints. Some constraints won't be detailed too much if it is not interesting to go deeper in the understanding of them. However, we will explain all of them and the ideas behind their pruning. For the last constraints (directed acyclic graph and path), we will have a deeper look into their functioning principles as they are very interesting and they use a lot of new concepts.

4.2 Graph binary relations

Before starting with the implementation of new constraints, we first have to define what we need to do to create a new constraint in *Oscar*.

In constraint programming, a constraint has to check that it is consistent to its definition and remove inconsistent values from the domains of its variables.

In *Oscar*, a constraint has to extend the abstract class `Constraint` and, to do so, we have to override two methods:

- the `setup()` method: this method is called when the constraint is posted. It checks that its semantic is correct, registers to potential domain modifications and launches initial propagation to prune domains.
- the `propagate()` method: this method is called when the domain of a variable linked to this constraint has been changed. Its role consists in:
 - detecting possible inconsistencies and, if it finds some, return `Failure`
 - detecting entailment and return `Success` after having detected it
 - removing values from the domain according to the filtering rules of the constraint and return `Suspend` after that

Readers willing to have more information about constraints and *Oscar* can find further detailed examples and explanations in the *oscar-cp* documentation [14].

Now that we know how to implement new constraints, let's first take a look at three binary relations between graphs, namely *SubGraph*, *Complement* and *Induced SubGraph*.

4.2.1 SubGraph

The `SubGraph(g1,g2)` constraint holds if g_1 is a subgraph of g_2 . To implement this constraint, we start first by defining a new class extending the `Constraint` class and we override the two methods `setup` and `propagate`:

```
class SubGraph(val g1 : CPGraphVar, val g2: CPGraphVar) extends
  Constraint(g1.s, "SubGraph") {
  override def setup(l: CPPropagStrength): CPOutcome = Suspend
  override def propagate(): CPOutcome = Suspend
```

The first step of the `setup` method will check if the current model is inconsistent. This inconsistency check is done by another method `inconsistent()` that will be explained later. Then it will register the two graph variables g_1 and g_2 to domain modifications in order to prune domains according to the constraint if there is a domain modification. Finally, it ends by launching the initial propagation to prune domains by calling the `propagate` method.

```
override def setup(l: CPPropagStrength): CPOutcome = {
  if(inconsistent) return Failure
  g1.callPropagateWhenDomainChanges(this)
  g2.callPropagateWhenDomainChanges(this)
  propagate()
}
```

The most interesting operations are the last ones and consist in the implementation of the `propagate` and the `inconsistent` methods. To be able to do them, we have to explain what bound consistency is, as well as the pruning that can be done on the graph variable domains. We can define the bound consistency of the `SubGraph` constraint as follows:

Theorem 4.1 *The $SubGraph(g_1, g_2)$ constraint is bound consistent if and only if the possible set of g_1 is included in the possible set of g_2 : $\overline{D}(g_1) \subseteq \overline{D}(g_2)$ and the required set of g_1 is included in the required set of g_2 : $\underline{D}(g_1) \subseteq \underline{D}(g_2)$.*

A more formal definition and a proof are given in [4]. This theorem allows us to establish a filtering algorithm for the constraint `SubGraph` on the graph variables g_1 and g_2 .

To do this we have first to check the inconsistency of domains; if $\underline{D}(g_1)$ is not a subgraph of $\overline{D}(g_2)$, then there is no solution because there is an element in g_1 that does not belong to g_2 and the constraint will fail. Then we can check entailment; if $\overline{D}(g_1)$ is a subgraph of $\underline{D}(g_2)$, then g_1 will always be a subgraph of g_2 and the constraint is entailed. Finally, we can prune domains. Each element in $\underline{D}(g_1) \setminus \underline{D}(g_2)$ should be included in $\underline{D}(g_2)$, as it is mandatory in

g_1 but not in g_2 . Each element in $\overline{D}(g_1) \setminus \overline{D}(g_2)$ should be removed from $\overline{D}(g_1)$, as it is not possible in g_2 . This algorithm is presented in 4.1.

```

Data:  $g_1, g_2$  two graph variables.
Result: Failure if inconsistency, Success if entailment and Suspend
           otherwise.
begin
  for  $x \in \underline{D}(g_1)$  do
    if  $x \notin \overline{D}(g_2)$  then
      | return Failure
    end
  end
  if  $\overline{D}(g_2) \in \underline{D}(g_1)$  then
    | return Success
  end
  for  $x \in \underline{D}(g_1) \setminus \underline{D}(g_2)$  do
    |  $\underline{D}(g_2) = \underline{D}(g_2) + \{x\}$ 
  end
  for  $x \in \overline{D}(g_1) \setminus \overline{D}(g_2)$  do
    |  $\overline{D}(g_1) = \overline{D}(g_1) - \{x\}$ 
  end
  Suspend
end

```

Algorithm 4.1: Illustration of the filtering of the SubGraph constraint

But it is also possible to see that the inconsistency might only happen when we post the constraint, in the `setup` method. Thanks to the pruning, it is indeed no longer possible to have an element in $\underline{D}(g_1)$ which is not in $\overline{D}(g_2)$ after the initial propagation. Therefore, the inconsistency is only checked once in the `setup` method and not at all in the `propagate` method. The implementation of method which will check the inconsistency is simply the first part of the algorithm 4.1.

The rest of the algorithm is the content of the `propagate` method. Initially and each time there is a domain modification, the `propagate` will check for the constraint entailment and prune domains. The implementation details are not presented here as they do not provide additional useful information. The implementation indeed simply uses the accessors of the `CPGraphVar` and loop over variables to prune or detect entailment as presented in algorithm 4.1. The code can be found in the class `SubGraph.scala`.

4.2.2 Graph complement

The `Complement(g1,g2)` constraint holds if g_1 is the complement graph of g_2 . If we remember the definition of a complement graph, each graph has the same number of nodes n , no edge of g_1 can be present in g_2 and symmetrically no edge of g_2 can be present in g_1 and all edges of the union of the edges of g_1 and of g_2 should form the complete graph with n nodes.

As we did in the `SubGraph` constraint, we have to create the class `GraphComplement` to implement the `Complement` constraint. We then have to override the `setup` method which has to register to domain modifications and to launch the initial propagation. Finally, we have to implement the method which is interesting here, namely the `propagate` method.

With the definition of complementary between two graphs, we can deduce the pruning rules of the `Complement` constraint. The `propagate` method which will implement these pruning rules will perform three operations. We will present these operations on one graph g_1 but they have to be done symmetrically on both graphs. The operations are the following ones:

Equal node sets In order to have a consistent `Complement` constraint, node sets should be the same. We therefore have to prune domains such that the upper bound of the nodes of both graphs should be the intersection of the upper bound of each graph and the lower bound of the nodes be the union of each lower bound.

Prune upper bound If an edge is required in one of the graphs, then this edge must not be possible in the other one. The corresponding pruning rule states that we must remove from $\overline{D}(g_1)$ all edges of $\underline{D}(g_2)$.

Prune lower bound If an edge is not possible in one of the graphs and if its endnodes are required, then this edge must also be required. The corresponding pruning rule states that we must add the complement of the edges in $\overline{D}(g_2)$ to $\underline{D}(g_1)$ if its endnodes are required.

The implementation is pretty straightforward and will not be detailed here. It is available in the class `GraphComplement.scala`.

4.2.3 Induced SubGraph

The `InducedSubGraph(g1,g2)` constraint holds if g_1 is an induced subgraph of g_2 .

Definition 4.1 g_1 is an induced subgraph of g_2 if and only if, for all pairs (u, v) of nodes of g_1 such that there exists an edge (u, v) in g_2 implies that there is an edge (u, v) in g_1 .

In order to be consistent, the definition implies that g_1 must be a subgraph of g_2 . In the `setup` method of the `InducedSubGraph` constraint, we have to post the `SubGraph` constraint, register to domain modifications and launch the propagation. Such an implementation looks like as follows:

```

class InducedSubGraph(val g1 : CPGraphVar, val g2: CPGraphVar) extends
  Constraint(g1.s, "InducedSubGraph") {
    override def setup(l: CPPropagStrength): CPOutcome = {
      if (s.post(new SubGraph(g1,g2)) == Failure) return Failure
      g1.callPropagateWhenDomainChanges(this)
      g2.callPropagateWhenDomainChanges(this)
      propagate()
    }
    override def propagate(): CPOutcome = Suspend
  }
}

```

Now, we only have to implement the `propagate` to prune domains and deal with the induced part of the constraint. As we know that g_1 is a subgraph of g_2 , we have to enforce lower bound of nodes to be the same in both graphs and enforce for each edge incident to two required nodes that if the edge is required in one graph, then it should also be required in the other. Conversely, if the edge is not possible in one, then it should also not be possible in the other one.

We can write a simple algorithm for this propagation, as can be seen on Algorithm 4.2. We only showed half of this algorithm here, given that the remaining is symmetric to the presented one. If we handle pruning by checking nodes and edges of g_1 compared with nodes and edges of g_2 , we indeed also have to do the reverse part. In the file `InducedSubGraph.scala`, we implemented the complete propagation algorithm.

```

Data:  $g_1, g_2$  two graph variables.
begin
  for  $n \in \underline{Nodes}(g_1) \setminus \underline{Nodes}(g_2)$  do
    |  $\underline{Nodes}(g_2) = \underline{Nodes}(g_2) + \{n\}$ 
  end
  for  $e \in \overline{Edges}(g_1)$  such that  $endNodes(e) \in \underline{Nodes}(g_1)$  do
    | if  $e \in \underline{Edges}(g_1)$  then
      | |  $\underline{Edges}(g_2) = \underline{Edges}(g_2) + \{e\}$ 
    | end
    | if  $e \notin \overline{Edges}(g_2)$  then
      | |  $\overline{Edges}(g_1) = \overline{Edges}(g_1) - \{e\}$ 
    | end
  end
  ... + symmetric pruning with  $g_1$  and  $g_2$  inverted
  Suspend
end

```

Algorithm 4.2: Illustration of the propaation of the InducedSubGraph constraint

4.3 Connectedness Constraints

The simple **Connected**(G) constraint holds if G is an undirected connected graph. We will start by explaining how to deal with undirected graph and the graph undirected connected constraint. As in all this master thesis, we are interested in the more general notion of directed graph. We will then end this section by distinguish the differences between weakly and strongly connected graphs which are constraints dealing with directed graph and their implementation.

4.3.1 Undirected connected graph

Before dealing with connectedness, let's define what the **Undirected** constraint is and how to implement it.

The **Undirected**(g_1, g_2) constraint holds if g_2 is the undirected graph obtained by ignoring the directions of the edges of g_1 . A filtering algorithm for this constraint first needs to check and to ensure that the sets of nodes of the two graphs are the same. After that, the filtering algorithm will prune g_1 by removing all the edges that are not present in g_2 from its domain and adding all the required edges from g_2 to g_1 . Finally, it will also prune g_2 . A possible edge of g_2 which is not possible in g_1 , and whose reverse counterpart is not possible in g_1 , will then be removed from g_2 . For each required edge of g_1 , we will add each edge and its counterpart to the required set of edges of g_2 . These pruning rules are summarized in the algorithm 4.3.

```

begin
  checkAndEqualNodesSets( $g_1, g_2$ )
  for ( $u, v$ )  $\leftarrow$   $\overline{Edges}(g_1)$ ; if ( $u, v$ )  $\notin$   $\overline{Edges}(g_2)$  do
    |  $\overline{Edges}(g_1) = \overline{Edges}(g_1) - \{(u, v)\}$ 
  end
  for ( $u, v$ )  $\leftarrow$   $\overline{Edges}(g_2)$ ; if ( $u, v$ )  $\notin$   $\overline{Edges}(g_1)$  do
    |  $\overline{Edges}(g_1) = \overline{Edges}(g_1) + \{(u, v)\}$ 
  end
  for ( $u, v$ )  $\leftarrow$   $\overline{Edges}(g_2)$ ; if ( $(u, v) \notin \overline{Edges}(g_1) \wedge (v, u) \notin \overline{Edges}(g_1)$ ) do
    |  $\overline{Edges}(g_2) = \overline{Edges}(g_2) - \{(u, v)\}$ 
  end
  for ( $u, v$ )  $\leftarrow$   $\overline{Edges}(g_1)$  do
    |  $\overline{Edges}(g_2) = \overline{Edges}(g_2) + \{(u, v)\}$ 
    |  $\overline{Edges}(g_2) = \overline{Edges}(g_2) + \{(v, u)\}$ 
  end
end
end

```

Algorithm 4.3: Example of pruning of the **Undirected** constraint

The **Undirected** constraint is implemented in the file **GraphUndirected.scala** and simply follows the presented pruning algorithm.

Now, we can focus on the graph undirected connected constraint. This constraint holds if an undirected graph g is connected. The filtering will be based on the three following properties:

Property 4.1 *If two nodes of $\underline{D}(g)$ belong to two different connected components of $\overline{D}(g)$, then there is no connected graph in $D(g)$.*

Property 4.2 *If $\underline{D}(g)$ is not empty, the union of all connected graphs in $D(G)$ is a connected graph.*

Property 4.3 *The intersection of all connected graphs in $D(g)$ is $\underline{D}(g)$ plus all the bridges and cutnodes on a path between two nodes of $\underline{D}(g)$.*

A proof of each property can be found in [4]. By using these properties, we can prune the bounds of the domain: if $\underline{D}(g)$ is not empty and if there is more than one connected component in $\overline{D}(g)$, then we should remove all the connected components that do not contain $\underline{D}(g)$. After that, we add all cutnodes and bridges in a path between two nodes of $\underline{D}(g)$ to $\underline{D}(g)$. An algorithm which summarizes the pruning rules of the `GraphUndirectedConnected` constraint is presented in Algorithm 4.4.

```

Data:  $g$  a graph variable.
begin
  if  $\underline{D}(g) \neq \emptyset$  then
     $CC \leftarrow \text{connectedComponents}$ 
    if  $\#CC > 1$  then
       $Creq \leftarrow \{x \in CC \mid x \text{ contains } \text{elem} \in \underline{D}(g)\}$ 
      for  $c \in CC \setminus \{Creq\}$  do
        for  $\text{elem} \in c$  do
           $\overline{D}(g) = \overline{D}(g) - \{\text{elem}\}$ 
        end
      end
    end
  end
  for  $n1, n2 \in \underline{Nodes}(g)$  do
    for  $\text{cutNodes}, \text{bridges} \in \text{path}(n1, n2)$  do
       $\underline{Nodes}(g) = \underline{Nodes}(g) + \{\text{cutNodes}\}$ 
       $\underline{Edges}(g) = \underline{Edges}(g) + \{\text{bridges}\}$ 
    end
  end
end

```

Algorithm 4.4: Illustration of the pruning rules of the undirected connected constraint

This algorithm was implemented in the file `GraphUndirectedConnected.scala`. However, the implementation details include more details than there are in the previous algorithm. We will develop some of them beneath.

When we compute the connected components and want to prune all the components that are not the one which contains $\underline{D}(g)$, an error might happen. If two connected components contain at least one element of the required domain, the constraint will indeed fail, as at least two required elements are disconnected. This might only happen in the first call of the propagate but it is important to notice and to handle it.

Beneath, we will also take a look at how the cutnodes and the bridges were computed. To detect cutnodes, we will check that the removal of a non-required node does not disconnect the graph. In fact, for each pair of nodes $n_1, n_2 \in \underline{D}(g)$, we will check whether if there is a path without going through a node in $\overline{D}(g) \setminus \underline{D}(g)$. If there is no path, then the node was a cutnode, as its removal would disconnect two required nodes. The node has to be added to the required set of nodes. The computation of the path starts from the first required node n_1 , gets all the neighbors of this node and iterate over them until n_2 is found or all the nodes of the graph have been explored. We will follow the same pattern with edges to detect bridges (we will check if there is a path between two required elements of the graph without going through a particular edge, etc).

Now that we have discussed the `UndirectedConnected` constraint for a graph variable, we will handle the directed version of it, namely the `DirectedConnected` constraint. But there exists two different notions of connectedness for a directed graph: the weak and the strong connectedness.

Definition 4.2 *A directed graph is strongly connected if every node of the graph can reach every other node.*

Definition 4.3 *A directed graph is weakly connected if the undirected version of the graph is connected.*

As we might have expected it, the weakly directed graph will use the `Undirected` and the `UndirectedConnected` constraints that we just explained and the strongly connected constraint will need to be completely implemented. However, we won't start from scratch since the pruning will use the same ideas as the ones used for the pruning of the undirected connected constraint.

4.3.2 Graph weakly connected

When we use the definition of the weakly connected graph, the constraint implementation is pretty straightforward. As a directed graph is weakly connected if its undirected version is connected, we simply have to create the undirected version of the graph and then post the `UndirectedConnected` constraint on it.

In practice, we first create a complete graph gu with the same number of nodes as the original graph g and we post the constraint `GraphUndirected(g,gu)`. By doing so, gu is now the undirected version of g . Then, we just have to post the `UndirectedConnected` constraint on gu .

The implementation is shown beneath and can be found in the class `GraphWeaklyConnected.scala`. The only difference between what was explained and the actual code source is the error handling.

```

class GraphWeaklyConnected(val g : CPGraphVar) extends Constraint(g.s,
  "Weakly Connected") {
  override def setup(l: CPPropagStrength): CPOutcome = {
    val gu = CPGraphVar(s, g.possibleNodes.size)
    if (s.post(new GraphUndirected(g,gu)) == Failure) return
      Failure // should not happen
    if (s.post(new GraphUndirectedConnected(gu)) == Failure) return
      Failure
    Suspend
  }
}

```

4.3.3 Graph strongly connected

The definition of the strongly connected graph is close to the definition of the undirected connected graph. The differences are that, in the undirected version, we can say that two nodes n_1, n_2 are connected if there is an undirected path from one to the other (we forget the direction of the edges); whereas the strongly connected graph definition, we need a directed path from n_1 to n_2 and from n_2 to n_1 to have the two nodes strongly connected.

But, as all other principles remain the same, we can use the algorithm 4.4 presented earlier in this thesis for the `Undirected` constraint. The differences between the previously explained algorithm and the algorithm which will handle the `StronglyConnected` constraint will be in the implementation when we need to compute the connected components, the cutnodes and the bridges. When we will search for a path between two nodes of the graph variable, the neighbors of the nodes will indeed only be the nodes reachable from the current one via a path through its outgoing edges. The `StronglyConnected` constraint was implemented in the file `GraphStronglyConnected.scala`.

4.4 No-Cycle Constraints

4.4.1 Directed acyclic graph

The `DirectedAcyclicGraph(g)` (*DAG*) constraint holds if g is a directed acyclic graph. In other words, it forbids any circuits in the graph g (a circuit being a path starting and ending at the same node).

This constraint is entailed if there is no circuit in the graph, and the constraint is inconsistent if there is a required circuit in the graph. The pruning rule for the constraint is the following: for any partial path in the required domain of g $\underline{D}(g)$, remove all edges from the possible domain $\overline{D}(g)$ that could create a required circuit if they were included in the required domain. We can also see this rule in the other way around; we can go through all possible not required edges $e : (e1, e2) \in \overline{D}(g) \setminus \underline{D}(g)$ and if we find a directed required path between $e2$ and $e1$, then we have to remove e from the possible domain because

adding e would later lead to inconsistency. When we talk about required paths (resp. circuits), we consider paths (resp. circuits) composed of required edges only, $edges \in \underline{D}(g)$.

From these rules, we can write a basic pruning algorithm for our **DAG** constraint. This algorithm is presented in Algorithm 4.5.

```

Data:  $g$  a graph variable.
Result: Success if entailment, Failure if inconsistency, Suspend
           otherwise.

begin
   $isCircuit \leftarrow false$ 
  for  $n_1 \in \overline{Nodes}(g)$  do
    if  $circuit(n_1)$  then
       $isCircuit \leftarrow true$ 
    end
  end
  if  $!isCircuit$  then
     $\text{return } Success$ 
  end
  for  $n_1 \in \overline{Nodes}(g)$  do
    if  $requiredCircuit(n_1)$  then
       $\text{return } Failure$ 
    end
  end
  for  $(u, v) \in \overline{Edges}(g)$  do
    if  $requiredPath(v, u)$  then
       $\overline{Edges}(g) = \overline{Edges}(g) - \{(u, v)\}$ 
    end
  end
   $\text{Suspend}$ 
end

```

Algorithm 4.5: Illustration of a basic pruning of the Directed Acyclic Graph constraint

We will now detail the implementation process. We first started with a naive idea: implement directly this algorithm 4.5. This implementation gave the expected results, but was not very efficient.

An improvement was proposed by Renaud Hartert. The idea was to use the graph structure and the properties of the **DAG** constraint to spare computations. The idea is based on the concept of transitive closure of a graph. We will explain here in details this solution and this concept because it will be also reused when we will deal with the **Path** constraint.

Let's remember the definition of transitive closure tc from section 2.2:

Definition 4.4 *The transitive closure of a graph $G : (N, E)$ is the graph $G^+ : (N, E^+)$ where E^+ is the set of edges: $\{(u, v) \in E \mid \text{exists a path from } u \text{ to } v\}$.*

If we remember the basic pruning algorithm, we compute the required path between each pair of nodes each time the pruning has to be done. We will now compute the required transitive closure to save computations. The goal will be to avoid to recompute a path between each pair of nodes each time the pruning is needed.

The graph representing the required transitive closure will have an edge between n_1 and n_2 if there is a required path between these two nodes. In practice, the transitive closure tc can be stored in a square matrix: for each pair n_1, n_2 of nodes, the value of the matrix $tc(n_1, n_2)$ will be set to 1 if there is a required path between n_1 and n_2 , otherwise the value will be set to 0.

With this required transitive closure, we are able to rewrite the pruning rules and improve overall computations. If n is a required node ($n \in \underline{Nodes}(g)$) and if the value of the transitive closure from n to n is equal to 1 ($tc(n, n) == 1$), then the constraint is inconsistent because there is a required circuit (we are able to make a circuit from node n). For each edge (u, v) which is part of the possible set of edges but which is not required ($(u, v) \in \overline{D}(g) \setminus \underline{D}(g)$), if the value of the transitive closure from v to u is equal to 1 ($tc(v, u) == 1$), then we have to remove the edge (u, v) because adding it in the required set of edges would lead to inconsistency. We can thus write the following algorithm 4.6 describing these new pruning rules:

```

Data:  $g$  a graph variable.
begin
   $tc \leftarrow buildTransitiveClosure$ 
  for  $n \in \underline{Nodes}(g)$  do
    if  $tc(n, n) == 1$  then
      | return Failure
    end
  end
  for  $(u, v) \in \overline{Edges}(g) \setminus \underline{Edges}(g)$  do
    if  $tc(v, u) == 1$  then
      |  $\underline{Nodes}(g) = \underline{Nodes}(g) - \{(u, v)\}$ 
    end
  end
  Suspend
end

```

Algorithm 4.6: Illustration of the pruning of the Directed Acyclic Graph constraint

This algorithm seems better than the previous one but we still have to build the transitive closure each time the pruning must be done. Building this transitive closure needs computing if there is a required path between each pair of nodes and computing the transitive closure entirely at each propagation still remains as costly as our basic algorithm.

Now, we will explain the significant improvement in terms of computations

proposed by Renaud Hartert. As we are only interested in the required transitive closure, we can improve significantly its computation: we will compute it entirely once and update it when needed. We have to compute the required transitive closure when we post the constraint and we have to update it by modifying the required values of this matrix only if it is needed and update only where a modification is susceptible to occur. We will divide this transitive closure update and modification in two steps which will bring the answers to the two following questions: when do we need to update the transitive closure and what do we have to modify.

To answer to the question of when we do need to update the transitive closure, we have to remember the four possible operations which can modify the domain of our graph variable: add an edge/node to the corresponding required set, remove an edge/node from the corresponding possible set. In fact, only one modification will modify the required transitive closure: add an edge to the required set of edges. This adding operation will allow more required paths to be built (as there is now one more required edge) and we have then to modify the transitive closure. To update *tc* only when an edge was added, we have to modify our `setup` method. Instead of using the standard registration as we did until now (`callPropagateWhenDomainChanges`), we can use a finer registration (called ‘level 1 registration’) to only detect when edges are added in the required set. To be able to do this, we have to add a new method in the `CPGraphVar`:

```
def callValRequiredWhenRequiredValue(c: Constraint) {
    E.callValRequiredWhenRequiredValue(c)
}
```

This method simply calls the level 1 registration method of the `CPSetVar` holding edges. We can now use this ‘level 1 registration’ in the `setup` method of the constraint `DirectedAcyclicGraph` to call the `ValRequired` method which has to update the transitive closure and launch the propagation. When using such registration, the `ValRequired` method will only be called when an edge is added. The `setup` method will also check inconsistency, as inconsistencies may only happen when the constraint is posted (the pruning will then prevent inconsistencies). This method is presented below:

```
override def setup(l: CPPropagStrength): CPOutcome = {
    for (i <- 0 to n-1; if tc(i)(i).getValue == true) return Failure
    // Level 1 registration to have more efficient pruning and update of tc
    g.callValRequiredWhenRequiredValue(this)
    propagate()
}
```

As we can see from the previous example, we stored the transitive closure matrix as a boolean matrix instead of a 0 and 1 matrix. In fact, it is not really booleans that are stored in the transitive closure, we used indeed `ReversibleBool` objects. The reason to use reversible elements is pretty obvious. In constraint programming, during the search when solving a *CSP*, we try to assign values

to variables and we have to revert the current state to a previous state if the current one does not lead to a solution. Now that we have decided to update the transitive closure and not build it at each propagation, we need to be able to restore to a previous transitive closure when restoring to a previous state. This is why we used `ReversibleBool` objects. They are indeed boolean values which allow us to backtrack to a previous state (and to recover the boolean values that formed the transitive closure at this previous state).

To finish, we still need to explain how we updated the transitive closure and how we did the pruning. The pruning follows in fact simply the second part of the algorithm 4.6, page 32: for each edge $(u, v) \in \overline{D}(g) \setminus \underline{D}(g)$, if $tc(v, u)$ is true, we remove (u, v) . This pruning is implemented in the `propagate` method and is called once by the `setup` method and called later by the `ValRequired` method each time an edge is added to prune domains accordingly. The last operation to be explained is the update of the transitive closure.

Even if the update would need to recompute each time the transitive closure entirely, it is still an improvement from the basic idea, as it is only called when an edge is added. But we can do better if we update only the required part of the transitive closure to save even more path computations. In order to achieve such an update, we have to look at what changes to bring in the current transitive closure tc when an edge is added to the required set. The simplest way to understand what we have to do is to use an example. On figure 4.1, we show one graph g and its required transitive closure on the left, an update of both of them on the right. On figure 4.1a, we can see a graph composed of five nodes, three required edges in plain lines and one possible but not required edge in dashed lines. The required transitive closure is shown in dotted lines. On figure 4.1a, we can see that the transitive closure corresponds to the three required edges, as the three required edges are disjoint. To show the update of the transitive when a modification occurs, we add the possible edge which stands in the middle of the graph to the required set. The result is shown on figure 4.1b. On this figure, we can see that the transitive closure in dotted lines contains the required path from each pair of nodes.

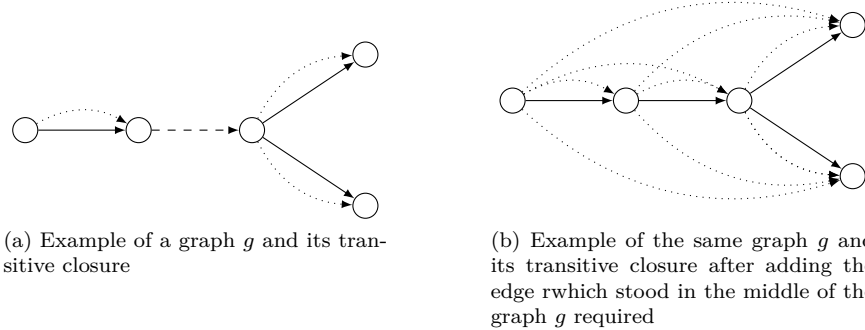


Figure 4.1: Example of the update of a graph and its required transitive closure

From this example, we can deduce an algorithm to update the transitive closure from a specific state to another. If we suppose that we added the edge (u, v) , we first have to modify tc to add the required path from u to v such that: $tc(u, v) = true$. Then we need to compute all nodes that have a path to u (we can do so by checking in the transitive closure if $tc(, u) == true$), and all nodes where v can go (in the transitive closure, nodes such that $tc(v,) == true$). Let us define nu the union of all nodes that have a path to u with the node u such that $nu : \{n \in \overline{Nodes(g)} | \exists path(n, u)\} \cup \{u\}$ and define vn the union of all nodes where v can go with the node v such that $vn : \{n \in \overline{Nodes(g)} | \exists path(v, n)\} \cup \{v\}$. For each pair of elements $(nu(i), vn(j))$ of nu , vn respectively, we have to set the value of the transitive closure matrix $tc(nu(i), vn(j))$ to true because adding the edge (u, v) has created a required path from $nu(i)$ to $vn(j)$. This may seem complicated at first sight but if we come back to our example of figure 4.1, this is not that difficult to understand. Let's assume that we noted nodes one to five from left to right, we have to add to nodes $\{1, 2\}$ (the union of the nodes that have a path to u and u) the nodes $\{3, 4, 5\}$ (the union of the nodes where v can go and v) in tc : $\forall u \in \{1, 2\}, v \in \{3, 4, 5\}, tc(u, v) = true$. If we do so, we update the transitive closure from figure 4.1a to figure 4.1b and we can check that the figure 4.1b verifies the definition of required transitive closure of the graph g .

The **ValRequired** method implements this transitive closure update that can be summarized in the algorithm 4.7. The only difference in the implementation in comparison with the algorithm is that we have to use the accessor **getValue** and the modifier **setValue** to interact with each element of the transitive closure as its elements are **ReversibleBool** objects.

Data: g a graph variable, tc the transitive closure, u, v the added edge.

```

begin
   $\overline{Edges(g)} = \overline{Edges(g)} + \{(u, v)\}$ 
   $from \leftarrow \{n \in \overline{Nodes(g)} | tc(n, u) == true\}$ 
   $to \leftarrow \{n \in \overline{Nodes(g)} | tc(v, n) == true\}$ 
  for  $i \in from$  do
    for  $j \in to$  do
       $tc(i, j) = true$ 
    end
  end
end

```

Algorithm 4.7: Illustration of the update of the transitive closure

4.4.2 Graph bipartite

A graph is bipartite if and only if its node set can be partitioned into two sets N_1, N_2 such that each edge has its endpoints in both sets: $E \subset (N_1 \times N_2) \cup (N_2 \times N_1)$. A bipartite graph has also the following property that will be used

in the filtering:

Property 4.4 *A graph is bipartite if and only if the length of each of its cycles is even.*

The `setup` method of this constraint will simply register the graph to domain modifications and call the `propagate` method. The `propagate` method will be in charge of detecting inconsistency, detecting entailment and pruning domains. For these three operations, we need to go through all nodes using a depth-first search and color them alternately by using two colors; the first node will be of the color 1, its neighbors of the color 2, their neighbors of the color 1,... etc. In practice, we can keep track of the node's color by adding the number corresponding to the node to a list of nodes having the same color. However, we have to take care about connected components. If we have two or more disconnected components in our graph, we have indeed to reason on each connected component independently. But to keep it simple here, we will only consider that there is a single component and we will leave the multiple connected components to implementation details (in fact, we simply need to store each component in a different list and prune accordingly).

To detect inconsistency, we can go through all the required edges and build the two color lists. Once this is done, we will be able to detect inconsistency if a node is present in both lists. If a required node has two different color, we can find an even length cycle and the constraint is inconsistent. To detect if a node is present in two lists, we can just look at the intersection between the lists and check if it is not empty. To detect entailment, we can follow the similar pattern but with the possible edges. We go through all possible edges and build the two color lists. If none of the nodes has two different colors, then the constraint is entailed because it is impossible to make an even length cycle. To prune domains, we check for each non-required edge if both of its endpoints can be in the same color set made by the required edges for inconsistency detection. If both endpoints can have the same color, then we have to remove this edge from the domain because if we add it later to required set, it would lead to inconsistency. All these filtering rules are summarized in the following algorithm 4.8.

We implemented this algorithm 4.8 in the method `propagate`. The constraint `Bipartite` was implemented in the file `GraphBipartite.scala`.

4.4.3 Tree

The `Tree` constraint holds if g is a undirected tree.

Definition 4.5 *g is an undirected tree if g is a connected acyclic undirected graph.*

A basic idea would be to post a `Connected` constraint and a `DAG` constraint on g to prune its domain. In fact, G. Doms has proven in [4] the following property:

Data: g a graph variable, tc the transitive closure, u, v the added edge.
Result: *Success* if entailment, *Failure* if inconsistency, *Suspend* otherwise.

```

begin
  color1, color2  $\leftarrow$  DFScoloring(Edges( $g$ ))
  if color1  $\cap$  color2  $\neq \emptyset$  then
    | return Failure
  end
  possColor1, possColor2  $\leftarrow$  DFScoloring( $\overline{\text{Edges}(g)}$ )
  if possColor1  $\cap$  possColor2 =  $\emptyset$  then
    | return Success
  end
  for  $(e_1, e_2) \in \overline{\text{Edges}(g)} \setminus \text{Edges}(g)$  do
    | if  $e_1 \in (\text{color1} \cap \text{color2})$  or  $e_2 \in (\text{color1} \cap \text{color2})$  then
    | |  $\overline{\text{Edges}(g)} = \overline{\text{Edges}(g)} - \{(e_1, e_2)\}$ 
    | end
  end
end
end

```

Algorithm 4.8: Illustration of the pruning of the Bipartite constraint

Property 4.5 *The pruning obtained by applying the pruning rules of the DAG and Connected constraints is bound consistent.*

Thus, we can prune domains by applying our basic idea. An implementation of the `setup` method is presented below. The complete code of the class can be found in the file `Tree.scala`.

```

override def setup(l: CPPropagStrength): CPOutcome = {
  if (s.post(new DirectedAcyclicGraph(g)) == Failure) return Failure
  if (s.post(new GraphWeaklyConnected(g)) == Failure) return Failure
  Suspend
}

```

4.5 Path Constraints

This section is dedicated to the well-known `Path` constraint. Let's remember the definition of a simple path from section 2.2.

Definition 4.6 *A path in a graph is a sequence of non-repeated nodes and edges starting from a node and ending at another node*

There exist many path problems in CP but two famous ones are the vehicle routing problem and the traveling salesman problem. These problems can be modeled as constrained path finding problems. To model them with our graph variable, we need a `Path` constraint.

We can distinguish two different kinds of `Path` constraints. The first one only handles paths between nodes (and focus thus on reachability) while the other one deals with weighted edges. We will here call the first one `SimplePath`, as it does not deal with weights, and the second one `ShorterPath`, as it allows us to search for a simple path whose total weight is lower than a bound within a directed weighted graph. We will now detail these two constraints, starting by the `SimplePath` constraint. The `ShorterPath` constraint will indeed re-use some of the pruning rules of the `SimplePath` constraint and we will thus only explain what was added to deal with the weight of the edges.

4.5.1 Simple path

The `SimplePath(g,n1,n2)` constraint holds if there exists a simple path from n_1 to n_2 in the graph g . In graph theory, a simple path is a directed acyclic connected graph. Thus, we will reuse some of the ideas developed for the `DirectedAcyclicGraph` constraint and for the `Connected` constraint here.

We will first explain the `setup` method which will be called when we post the `SimplePath` constraint. When we post this constraint, we add the source and the destination of the path to the required set of nodes; the model is indeed no longer consistent if we remove one of them during the search. Then, we remove all the incoming edges to the node which start the path as well as all the outgoing edges from the destination, as they should not be used in the graph. After this first pruning, we can register to domain modifications to call the `propagate` method which launches the initial propagation. The implementation of this `setup` method is pretty straightforward and detailed right below.

```
override def setup(l: CPPropagStrength): CPOutcome = {
  if (g.addNodeToGraph(src) == Failure) return Failure
  if (g.addNodeToGraph(dest) == Failure) return Failure

  for (e <- g.possibleInEdges(src)) if (g.removeEdgeFromGraph(e) ==
    Failure) return Failure
  for (e <- g.possibleOutEdges(dest)) if (g.removeEdgeFromGraph(e)
    == Failure) return Failure

  g.callPropagateWhenDomainChanges(this, false)
  propagate()
}
```

We will now explain the most interesting part: the content of the `propagate` method. The constraint is entailed if there is a required path in the graph g . A required path is a path composed of required edges. Checking entailment will be the first step of the `propagate` method. We will then prune the domain of the graph according to the constraint definition. The pruning rules for this constraint can be split into four aspects: remove the nodes and remove the edges that should be forbidden to respect the constraint definition from the corresponding possible set, add the nodes and add the edges that should be

mandatory to the corresponding required set. We will now explain each one after the other.

Forbidden edges To filter the edges, our pruning will be based on the following properties given in [4]:

Property 4.6 *Let $TC(\overline{D}(g))$ denote the set of edges in the transitive closure of $\overline{D}(g)$. An edge $(u, v) \in \overline{D}(g)$ is forbidden if*

$$(n_1, u) \notin TC(\overline{D}(g)) \vee (v, n_2) \notin TC(\overline{D}(g)) \vee \\ \exists n \in TC(\underline{D}(g)) : (n, u) \notin TC(\overline{D}(g)) \wedge (v, n) \notin TC(\overline{D}(g))$$

This property states that each starting point of an edge should be reachable from the source of the path, that each ending point of an edge should reach the end of the path and that the edge can not skip a required node. An illustration of this property is shown on figure 4.2. In this example, we can also see that three edges should be removed. If we use the edge from 1 to 3 in a path, then we will skip the required node 2 in this path. The pruning will then remove (1, 3). Also, we have to remove the edges (4, 2) and (2, 5) because there is no path from n_1 to 4 and from 5 to n_2 respectively.

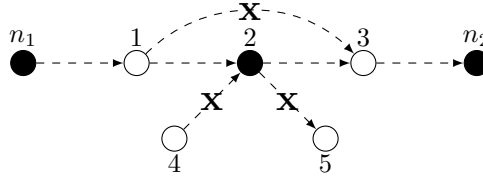


Figure 4.2: Forbidden edges

We can simply prune $\overline{D}(g)$ with this property and with the transitive closure of the possible domain. We can check that the property holds for each edge and remove the edge if it does not. The pruning will check that the source of the edge is reachable from the source, that the destination of the edge can reach the path destination and that the edge does not jump over a required node in the path.

But we still have to define how we compute $TC(\overline{D}(g))$ and how we update it during the search. The answer to those questions will be provided further in this thesis when we will explain the implementation details.

Mandatory nodes To test if a node should to be required in a graph variable g , we can remove each node one after the other and check if its removal prevents n_1 from reaching this required node or prevents this required node from reaching n_2 . If the removal of this node disconnects the graph, the node must be required and thus added to the required set of nodes.

Mandatory edges The required edges are the bridges. If we detect a bridge and that its removal disconnects at least two required elements of $\underline{D}(g)$, then this edge which is a bridge must be added in the required set of edges.

Forbidden nodes Each isolated node is forbidden by definition. During the search, if a node is not the destination of the path and does no longer have outgoing edges, then this node must be removed.

We can write an algorithm to prune domains of our **SimplePath** constraint using these rules. This algorithm is presented in Algorithm 4.9.

```

Data:  $g$  a graph variable.
Result: Success if entailment, Failure if inconsistency, Suspend
           otherwise.
begin
   $c \leftarrow isEntailed$ 
  if  $c \neq Suspend$  then
    |  $return\ c$ 
  end
  //Forbidden edges
  for  $(u, v) \in \overline{Edges}(g)$  do
    | if  $!tc(n_1, u) \vee !tc(v, n_2)$  then
    | |  $\overline{Edges}(g) = \overline{Edges}(g) - \{(u, v)\}$ 
    | end
    | for  $n \in \underline{Nodes}(g)$ ; if  $!tc(n, u) \wedge !tc(v, n)$  do
    | |  $\overline{Edges}(g) = \overline{Edges}(g) - \{(u, v)\}$ 
    | end
  end
  //Mandatory nodes
  for  $n \in \overline{Nodes}(g) \setminus \underline{Nodes}(g)$  do
    | if  $checkMandatory(n)$  then
    | |  $\underline{Nodes}(g) = \underline{Nodes}(g) + \{n\}$ 
    | end
  end
  //Mandatory edges
  for  $(u, v) \in \overline{Edges}(g)$ ; if  $bridge(u, v)$  do
    |  $\underline{Nodes}(g) = \underline{Nodes}(g) + \{n\}$ 
  end
  //Forbidden node
  for  $n \in \overline{Nodes}(g)$ ; if  $outEdges(n) = \emptyset$  do
    |  $\overline{Nodes}(g) = \overline{Nodes}(g) - \{n\}$ 
  end
   $Suspend$ 
end

```

Algorithm 4.9: Illustration of the pruning of the Simple Path constraint

We will now explain the implementation of the `SimplePath` constraint, as well as its practical details and improvements of its pruning in term of complexity.

We starts with the entailment of the constraint. The `isEntailed` method will check if there is a required path in the graph. In order to achieve that, it will perform a depth-first search from the source node towards the destination node but, as it must be a required path, the neighbors from a node will be the nodes reachable using a required edge. If there exists a required path, then all the other edges are removed because they are no longer useful and may lead to inconsistency. This method and then the calling method, the `propagate` method, will return *Success* after that.

Another interesting aspect to be explained here is the transitive closure construction and the update that will be used to prune $\overline{Edges}(g)$ (we can remember the definition of transitive closure from section 2.2 and from the definition of the constraint `DAG`). At first sight, there are similarities with the approach we used for the `DAG` constraint but the two are in fact very different. For the `DAG` constraint, we used a transitive closure build with the required path(s) between each pair of nodes. It was then used to prevent circuits. For the `Path` constraint, we will build a transitive closure using possible path(s) between two nodes, as we need $TC(\overline{D}(g))$ to prune domains. The transitive closure used here will therefore be different from the one used for the `DAG` constraint.

The transitive closure will be stored here in an square matrix composed of `ReversibleBool` objects where $tc(u, v)$ is equal to `true` if there exists a path from u to v and equal to `false` otherwise. The reversible part will allow backtrack to a previous state in the search without recompute the transitive closure. With an initial matrix filled with `true` (we assume initially that there is a path between each pair of nodes), this update procedure will consist in checking if there is a path and in updating the transitive closure tc accordingly. Such an update is summarized in Algorithm 4.10. To use this algorithm, we had to implement the `existPath` method. Without going into unessential details, the method `existPath` simply performs a depth-first search in the graph from the source to find the destination node. Furthermore, the update of this transitive closure that will be called within the propagation also check for each pair of node if there is a path but without doing useless computation. It first checks the value stored in the transitive closure. If the value stored in $tc(n_1, n_2)$ is already `false` (there is no path between n_1 and n_2), then there is no need to go further because a path can not be created. On the other hand, if $tc(n_1, n_2)$ is `true`, then the update will call the method `existPath(n1,n2)` to check if a path is still possible and will update the transitive closure tc accordingly.

We can however improve drastically its efficiency of this transitive closure. Even if a single depth-first search does not take much time, the way the transitive closure is currently built can indeed really become an issue in terms of complexity. We will now consider an improvement to the transitive closure update that will significantly reduce the number of calls to the method `existPath`. The idea is to re-build the transitive only when it is needed. Adding an edge or a node to its required set does indeed not modify the transitive closure, as it does not interfere with the possible path(s). We will then update the transitive

```

begin
  for  $n_1, n_2 \in \overline{Nodes}(g)$  do
    if !  $existPath(n_1, n_2)$  then
      |  $tc(n_1)(n_2).setValue(false)$ 
    end
  end
end

```

Algorithm 4.10: Illustration of the construction of the transitive closure tc

closure only when a node or an edge is removed. To update tc only when a node or an edge was removed, we store the number of possible nodes and the number of possible edges as two **ReversibleInt** objects. If the current value is lower than the stored one (which means that an element was removed), we have to update the transitive closure because some possible paths may no longer be possible.

As previously mentioned, the pruning of the edges upper bound $\overline{Edges}(g)$ will use tc as defined right before. But we can also improve the pruning by removing additional possible edges. To do that, we remove the reverse $((v, u))$ of each required edge $(u, v) \in \underline{Edges}(g)$, all the other edges starting from the source of these edges $(u,)$ and all the other edges which have the same destination of the destination than these required edges $(,v)$. For each edge $(u, v) \in \underline{Edges}(g)$, we can remove, if it is present in the graph variable g , (v, u) the reverse because it has no sense to use it if (u, v) is required and we can remove all edges $(u,)$ as the edge starting from u is (u, v) and all edges $(,v)$ as the edge going to v is (u, v) .

Having detailed the first pruning rule, we will now focus on the three remaining pruning rules described in Algorithm 4.9. The second pruning rule adds to $\underline{Edges}(g)$ all the edges that are bridges between the source and the destination of the path. To achieve such a pruning, we simply look for all the required nodes and check their outgoing edges. If there is only one outgoing edge, this outgoing edge is a bridge (as we have to go through this required node in the path and that there is only one outgoing edge to this node). We then must add this edge to the required set of edges.

The third pruning rule consists in removing nodes that are isolated. This is pretty straightforward, as we just have to check for all the nodes, except the destination, that these nodes have no outgoing edges. In such a case, the node has to be removed as it will not contribute to any path from the source to the destination.

The last pruning rule adds all the nodes that are cutnodes to the required set of nodes $\underline{Nodes}(g)$. A way to compute the cutnodes is, for each possible node, to ignore this node and search for a path which goes from the source to every required node and another which goes from each required node to the destination. If no such path is found, we have to add this node to the required

set, as this node is required to have a path from the two required nodes (the source and the destination are indeed required by definition). To do so, we implement a method `checkMandatoryNodes` that check if all the required nodes are still reachable from the source and that all the required nodes can still reach the destination without going through a particular node. In practice, the reachability is simply a depth-first search where the node to ignore is removed from the neighbors of each node. We will then search for a path without going through this particular node.

4.5.2 Shorter path

This section will be dedicated to the shorter path constraint. We decide here to call this constraint `Path` to keep it simple and brief.

The `Path(g, n1, n2, W, I)` constraint holds if g is a simple path from n_1 to n_2 whose total weight is not greater than I . The total weight of the path is computed as the sum of the weights of every edge in the path. The function w maps each edge with its weight as $(Int, Int) \rightarrow Int : (u, v) \in \overline{Edges}(g) \rightarrow w(u, v)$. The bound I will be the bound of the total weight of a path. In other words, I will be a range of values $I \in [I_{min}, I_{max}]$ such that I_{min} is the minimal path length from the source to the destination and I_{max} is the maximal path length lower or equal to the maximal bound.

This constraint is an adaptation of the shorter path constraint for the boolean arc model presented in [17] and corresponds to the constraint described in [4]. Achieving arc consistency for shorter path constraint is NP-hard. The pruning of this constraint will then achieve a relaxed consistency. Some of the pruning of this `Path` constraint will be the same as the `SimplePath` constraint but we can do additional pruning with the weight of the edges and the bound of the path length.

We will detail the `Path` constraint, its implementation and the pruning rules associated with this constraint. We start with the initialization of the constraint. The `setup` method for this constraint is the same method as the `SimplePath` one. The initialization indeed adds the source and the destination of the path to the required set of nodes. It will also remove all the incoming edges to the starting node of the path and all the outgoing edges from the destination node. Finally, it will register to domain modifications and launch initial propagation by calling the `propagate` method.

Pruning rules for this constraint will now involve the weight of the edges and will allow us to model and face more interesting problems. However, the pruning rules used for the `SimplePath` constraint remain relevant. The pruning rules for the `Path` constraint will then become the union of the pruning rules of the `SimplePath` with an additional pruning rule based on the weights of the edges. We will detail this pruning rule now, as the `SimplePath` pruning rules have already been explained in the previous section.

The pruning rule will be based on the consistency property:

Property 4.7 *All the edges of $\overline{D}(g)$ belong to at least one path in $\overline{D}(g)$ from n_1 to n_2 with cost no greater than $\overline{D}(I)$.*

From this property, we can deduce the following pruning rule. First, we have to compute the shorter single source path from n_1 to all nodes of g , let's call these distances $d(n_1, n) \forall n \in \overline{Nodes}(g)$. Then, we have to compute the same kind of distance but from all nodes to the destination of the path n_2 , let's call them $d(n, n_2)$. Using these distances, we can prune the possible edges $\overline{Edges}(g)$ as presented in [18] and [4]:

an edge $(u, v) \in \overline{Edges}(g)$ is forbidden if $d(n_1, u) + w(u, v) + d(v, n_2) > \overline{D}(I)$

Intuitively, this pruning rule states that an edge should be removed if there is no path from n_1 to n_2 going through (u, v) such that its corresponding length is lower than $\overline{D}(I)$. Before detailing the implementation and practical issues, we still have one operation to perform in the body of the **propagate** method: we have to update the value of the bound I . When solving a *CSP*, we want to have I as close as possible to the optimal path length to have the best possible pruning. In order to do that, we have to update I_{min} and I_{max} carefully to prevent to remove the optimal solution but also in a smart way to prune domains and update this bound efficiently. We will update I such that its lower bound is lower than the optimal solution and the upper bound higher.

All the pruning rules and update operations are summarized in the Algorithm 4.11. We will now detail the implementation of these rules and updates.

```

begin
  for  $n \in \overline{Nodes}(g)$  do
     $d(n_1, n) = \text{computeDist}(n_1, n)$ 
     $d(n, n_2) = \text{computeDist}(n, n_2)$ 
  end
  for  $(u, v) \in \overline{Edges}(g)$  do
    if  $d(n_1, u) + w(u, v) + d(v, n_2) > I_{max}$  then
       $\overline{Edges}(g) = \overline{Edges}(g) - \{(u, v)\}$ 
    end
  end
  end
  /*
   * Same Pruning rules as SimplePath
   */
   $I_{min} \leftarrow \max\{I_{min}, \text{minimalDistance}(g)\}$ 
   $I_{max} \leftarrow \min\{I_{max}, \text{maximalDistance}(g)\}$ 
end

```

Algorithm 4.11: Illustration of the propagation of the **Path** constraint

To implement the pruning rule, we have to compute the shorter distances between the source node and all the possible nodes of the graph and to compute

the shorter distances between all the possible nodes and the destination. We implement the famous Dijkstra algorithm to obtain all the distances from the source to the other nodes. We can do the same to have the shortest distance from all the possible nodes to the destination and compute the Dijkstra algorithm from all the possible nodes and stop when we reach the destination. To be more efficient than computing several times the Dijkstra algorithm, we can compute only one time the Dijkstra algorithm from the destination to all the possible nodes to have the distances. But the problem here is that the graph is directed and an edge (u, v) and its reverse (v, u) are different. To solve this problem, we compute a variant of the Dijkstra algorithm: we start from the destination and we go through all in-neighbors of the destination. The in-neighbors of a node i are the nodes such that there is an edge $(-, i)$. Using this neighbors definition, we can iterate over the nodes to find all the distances from all the nodes to the destination.

The implementation of this pruning rule and the associated Dijkstra computations can be found in the file `Path.scala`. However, we will have to detail the update of the bounds of I .

A first idea to update the value of the lower bound I_{min} is to compute the sum of the shortest outgoing edge of each required node, except the destination which has no outgoing edges. Intuitively, this sum will be lower than the shortest path because every required node must have at least one outgoing edge (except the destination) and if we take the edge with the shortest associated weight, it will be lower or equal to the weight of the outgoing edge of this node chosen for the optimal value of the shortest path. We can use a similar idea to update the value of the upper bound I_{max} . The new upper bound value will be equal to the minimal value between the current bound value and the sum of the weight of the maximal outgoing edge of each possible node. With such an update, the constraint `Path` is now working but not very efficient. These naive updates do indeed not provide a good interval I and consequently do not achieve a good pruning.

We will now detail the improvement for the update of the bounds. Research has been made in Graph Theory to find the most efficient bound for the shorter path problem. G. Brassard and P. Bratley showed in [19] that the minimal spanning tree of a complete graph is a lower bound of the Hamiltonian cycle. An Hamiltonian cycle is a cycle in a graph that visits each node exactly once. In fact, our definition of shortest path is a tree because there is no cycle and the minimal spanning tree is thus a lower bound of our tree. This definition is not completely true because we must only consider the required nodes of the graph (i.e. build a minimal spanning tree with these required nodes) and we could have disconnected required components. When we talk about disconnected components, we are talking about components that have no connection between them using only required edges.

We implemented a variant of the well-known Kruskal algorithm with these considerations about connectedness and required nodes to obtain a lower bound of the shortest path. The standard Kruskal algorithm is presented in [19]. The update of the lower bound I_{min} now computes the minimal spanning tree with

all the required edges and sums the weight of each edge in the minimal spanning tree. Such an update is way more efficient than the presented basic idea because it computes a bound closer to the shortest path length.

Literature provides many approximations of the upper bound of the shortest path. A first and simple approximation of the upper bound is equal to two times the value of the minimal spanning tree weight. A better one that is used for the travel salesman problem but which could be modified to fit the constraint here is the Christofides algorithm which provides a bound which is $3/2$ of the value of the optimal shortest path weight. The idea that was implemented in the **Path** constraint to compute an upper bound was a trade-off between the time needed to compute the bound and the efficiency of this bound. In fact, the bound approximation which is equal to two times the minimal spanning tree weight is very cheap in term of computation as the minimal spanning tree weight is already computed for the lower bound. This approximation is also very efficient at the start of the search as we does not have found a solution yet. But later, when several edges are required, the first approximation which computes the bound as the sum of the maximal outgoing edge weight of each required node becomes better. The implemented approximation will compute the minimum of these two values to have the strengths of both ideas at low cost. Let *maxL* be the sum of the weights of each maximal outgoing edge of each required node and let *minBound* be the minimal bound computed as the minimal spanning tree weight, the update of the upper bound will be computed as: $maxBound = \min(maxL, 2 \times minB)$.

Chapter 5

Applications and experiments

In this chapter we will use our graph variable and some constraints to test the correctness and performance of our implementation.

All the pruning rules that were presented in the previous chapter were tested. These tests are available in the project repository [6] in the package `oscar-cp-test`. However, a real problem brings better information about correctness and efficiency. We then chose to focus on the famous Traveling Salesman Problem *TSP*. We will solve instances of this problem using the graph model and the integer model, and we will compare the results we obtained with the graph model with the ones we obtained with the standard integer model. Additional applications and examples of models using graph variables in constraint programming can be found in [1] and in [2].

5.1 Travel Salesman Problem

In the Traveling Salesman Problem, a salesman has to visit a series of cities. The problem to solve is to find the shortest route going through each city only once and returning to the first city at the end.

We will first explain the modeling of this problem that is done without graphs. This model was implemented in `OscaR` by R. Hartert and P. Schaus. We will then explain a graph model of the *TSP* which will use our `CPGraph` and the constraint `Path`. Finally, we will compare the two approaches and discuss the results in term of pruning and time complexity.

5.1.1 Integer model

The standard integer model of a *TSP* in constraint programming states that each node must have a successor and that these successors must be different. In other words, we have to find the minimal assignment of cities/successors such

that there is a circuit which starts from a source city, goes through each city at least once, and ends at the original source city. Here we will explain the resolution of the Traveling Salesman Problem using the integer model without entering in details. A more formal explanation of this modelization can be found in [20].

To be able to model and solve the traveling salesman problem we have to provide the model a distance matrix *distMatrix* which represents the distance between each node of the graph, with *distMatrix(i)(j)* being the distance between the nodes *i* and *j*.

The variables of this model are the successors of the current node - the next city that the salesman have to visit - and the total distance of the circuit:

```
val succ = Array.fill(nCities)(CIntVar(Cities))
val totDist = CIntVar(0 to distMatrix.flatten.sum)
```

Each successor and the total circuit distance are instances of a `CIntVar` object. These values must indeed be modified during the search and return to a previous value in case of backtracking.

To model our *TSP*, we need to state that there must be a circuit going through all nodes such that the total distance is the sum of all edges in the path. The constraints that will link these variables in the travel salesman problem are the following ones:

```
add(circuit(succ), Strong)
add(minAssignment(succ, distMatrix, totDist))
add(sum(Cities)(i => distMatrix(i)(succ(i))) == totDist)
```

We will now detail these constraints one after the other.

circuit(succ): enforces a circuit representation where *succ(i)* represents the city visited after the city *i* (no city is thus visited twice and there are no sub-tours)

minAssignment: ensures that all successors are different and that the sum of the weight of each pair (*i, succ(i)*) is lower or equal to *totDist*

totDist: sets the total distance of the circuit as the sum of each pair (*i, succ(i)*) in the path

We can now search in the search space to find the solution that minimizes the total distance:

```
minimize(totDist) search binaryStatic(succ)
```

We can notice that we used the `binaryStatic` search which may not be the best one but which will allow us to compare this model with the model with graphs. We will refer to this model as *TSP* in the rest of this work.

The search can be greatly improved using a heuristic. One of the most famous and best ones is the first-fail heuristic. This heuristic chooses the variable whose domain is most likely to fail. It was designed and implemented in *OscAR* and its usage looks as follows:

```
minimize(totDist) search binaryFirstFail(succ)
```

In the rest of this work, we will refer to this integer model with a first-fail heuristic as *TSPFF*. The implementation of the integer model and the corresponding source file are available in annexe A.2.

5.1.2 Graph model

We will now use our graph variable to model the traveling salesman problem. Once again, we have to provide the model with the distance matrix *distMatrix* which represents the distance between each city.

We now have to ensure that there is a circuit between all the nodes of the graph and we want to use the **Path** constraint. We can first post that all the nodes of the graph should be required because we do not want to miss and/or remove a single city of our graph. We can then use the idea presented in [21] and in [3]: to model a *TSP* with a **Path** constraint, we will duplicate the starting node *s* (let us call this duplicate node *s'*) and we will post a **Path** constraint from the source node to this duplicate *s'*. It is easy to see that such a path corresponds to a circuit.

Assuming that we have updated our distance matrix with the duplicate node and that the starting node is the node 0, we can then directly create the variables of our model and post the constraints:

```
val totDist = CPIntVar(0 to distMatrix.flatten.sum)
val graph = CPGraphVar(nCities+1)
// all nodes in the path are required
for (n <- 0 to nCities) add(graph.addNode(n))
// path from 0 to duplicate n
add(new GraphPath(graph,0,nCities,dist,totDist))
```

We now have to search in the space to find the optimal solution. We use a branch-and-bound approach: we select a possible edge and either we add it to the required set, or we remove it from the possible set. If there is no more possible edges, then there are no alternatives.

Assuming that the variable *possibleEdges* contains a list of possible edges that are not required: $e \in possibleEdges \Leftrightarrow e \in \overline{Edges}(g) \setminus Edges(g)$, and that the method *selectFirst* selects the first edge of this list, then the code below illustrates the searching procedure. In the rest of this work, we will refer to this graph model as *TSPGraph*.

```

minimize(totDist) search {
  if (possibleEdges.isEmpty){ noAlternative }
  else {
    val (s,d) = selectFirst(possibleEdges)
    branch { add(graph.addEdge(s,d)) }
           { add(graph.removeEdge(s,d)) }
  }
}

```

Even if this model is perfectly working, it is nevertheless not very efficient. We will now detail the modifications we made to the model and to the searching procedure to improve it significantly.

We can first improve the searching procedure by modifying the selecting method. This method indeed selects the first edge of the list of possible edges, which is not very efficient. A better heuristic would be to select the edge which has the lowest weight among the possible edges. Using this heuristic, we can hope to find rapidly a short path and then prune the domain of our graph accordingly with the bounds of the total path distance.

This heuristic improves the search but we can do even more. This heuristic is indeed a good one but a drawback can be shown easily using an example, as can be seen on Figure 5.1. We have a graph g on Figure 5.1a. Applying the heuristic for the search of a path, we can see in Figure 5.1b the search after the two first iterations. The graph is now composed of two required edges which are the shortest ones. The orientation is random, the search gets the shortest edge and if there are two edges with the same weight, then it will just pick one of them. The graph is also pruned after each step. The possible edges are not shown in these figures to focus on the edges taken. We can see the third step of the search on Figure 5.1c where the shortest edge at this stage among the possible ones was added to the required set. The last figure, Figure 5.1d, shows when the first path is found, at the fifth step.

We can clearly see from this example that, even if the heuristic gives better results than picking edges randomly, the choices are not always the best ones. To improve this heuristic, we will build a path incrementally. The new heuristic will select the next edge to add to the required set as the edge with the lowest weight which source node is the current end-node of the path. In other words, the edge to select will be the shorter weight edge of the current end-node of the path. The new search algorithm will start from the source node and select the edge with the lowest weight from the source of the path. It then iterates from the end of the path which is currently build and select the next edge. The path build at this stage corresponds to the required path from the source to a node.

Assuming that `getEndPathNode` gives the end node of the required path starting from the source, the code below illustrates the searching procedure. The method `getEndPathNode` is simply a depth-first search from the source node through required edges only. At the end it will return the node ending the required path (i.e. the node which does not have outgoing required edges).

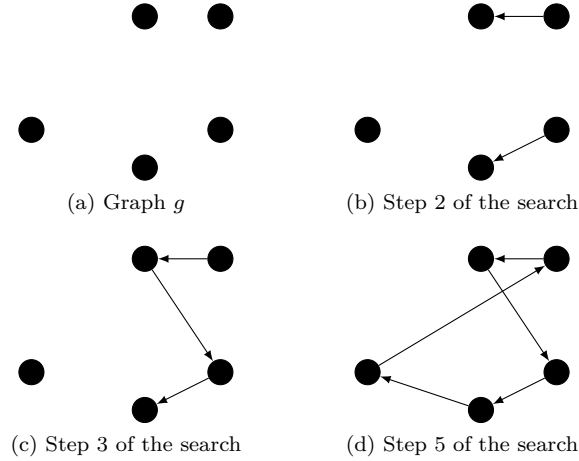


Figure 5.1: Illustration of a drawback of the heuristic only selecting the edge which has the lowest weight of the graph

```

minimize(totDist) search {
  if (possibleEdges.isEmpty){ noAlternative }
  else {
    val endNode = getEndPathNode // end of required path
    // and get the closest edge
    val (s,d) = selectMin(graph.possibleOutEdges(endNode))

    branch { add(graph.addEdge(s,d)) }
           { add(graph.removeEdge(s,d)) }
  }
}

```

Assuming that the source node is the one on the left, the figure, Figure 5.2, shows the first solution found by the search with the improved heuristic of the example shows on Figure 5.1. We can see here that this heuristic gives better results than the previous one. The search will nevertheless still have to explore the search space to find the optimal solution but this heuristic allows to find quickly a good solution and to prune the domains more efficiently.

The above presented heuristics improve the search, but we can also improve the model if we use such heuristics. In general, the choice of the source of the path is not important in the graph model. We indeed have to find a circuit to solve the *TSP* and the optimal solution will be the same, whatever the source node is. But if we use the heuristic search, starting with some nodes can be more efficient than starting with other ones. Starting from two different nodes can lead to two different heuristic circuits and we would like to take the shortest one to have a better first solution and consequently a better search and pruning.

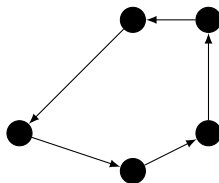


Figure 5.2: Illustration of the first solution of the example shown on Figure 5.1 found by the improved heuristic

We will therefore choose the starting node as the node leading to the shortest heuristic circuit length. We duplicate this node, post the constraint `Path` from this node to the duplicate one to model the *TSP*, and the heuristic search will lead to more efficient pruning. In the rest of this work, we will refer to this graph model with the previously explained heuristics as *TSPGraphH*. The implementation of both graph models and their corresponding source files are available in appendix A.3.

We will now compare the integer model with the graph model.

5.1.3 Comparison

Here we will build random *TSP* instances for 5 to 30 cities following the construction made by R. Hartert and P. Schaus in *Oscar* [5]. The instances on which were run the tests are available in appendix A.1.

We run¹ our models on the instances several times to have average values of the number of nodes explored and the elapsed time. Now we will compare the obtained results. We will first compare the basic integer model *TSP* with the basic graph model *TSPGraph*. We will then compare the integer model using the first-fail heuristic *TSPFF* with the graph model using heuristics *TSPGraphH*. We finally compare all these models together. When we talk about number of nodes explored in the various comparisons below, we compare the number of nodes in the search space that are explored to find the best solution.

TSP and TSPGraph

We can see on Figure 5.3 that the number of nodes explored in the basic graph model *TSPGraph* is bigger than the number of nodes explored in the basic integer model *TSP* for all the tested instances. However, we can also see that the evolution of the curve in the graph model seems promising. When the number of cities increase, the curves indeed come closer to each other. For example, the ratio between the number of nodes explored in the *TSPGraph* model and the *TSP* model is around 750 for 15 cities and decrease to around 25 for 20 cities. But we could not go further into this analysis because the computation time became too long to run the *TSPGraph* model on 25 cities.

¹The tests were run on a Intel(R) Core i7-4700HQ CPU @ 2.4GHz.

We can also see on Figure 5.4 that the curves representing the computation time in both models have the same shape as the curves of the number of visited nodes.

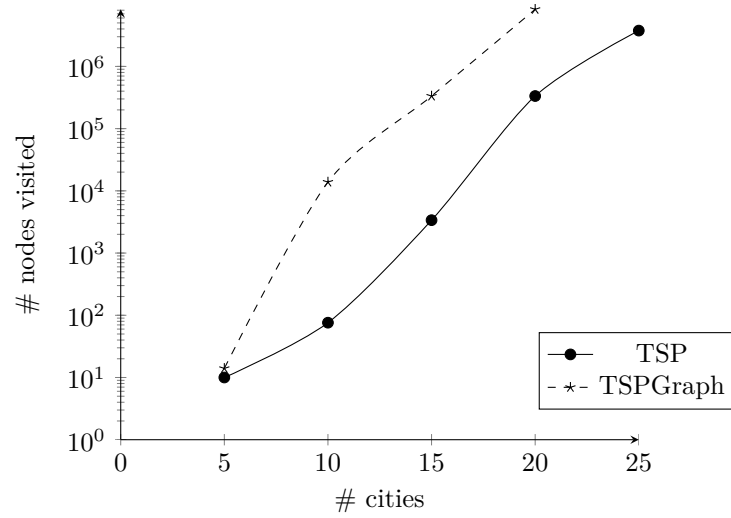


Figure 5.3: Illustration of the number of nodes visited in the two models: *TSP* and *TSPGraph*

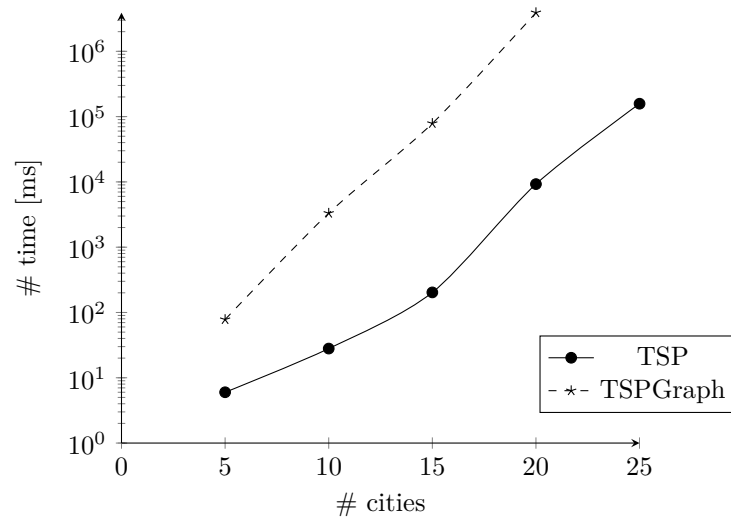


Figure 5.4: Illustration of the computation time in the two models: *TSP* and *TSPGraph*

TSPFF and TSPGraphH

We can see on Figure 5.5 that the number of nodes explored in the graph model with the heuristics *TSPGraphH* is bigger than the number of nodes explored in the integer model with the first-fail heuristic *TSPFF* for small instances. However, when we run the models for 20 and 25 cities, the number of nodes explored in the *TSPGraphH* model is smaller than the number of nodes explored in the *TSPFF* model. When the number of cities increase, the graph model gives better results because it can prune domains more efficiently using the structure of the graph.

But we could not go further into this analysis because the computation time became too long to run the *TSPGraphH* model on 30 cities.

We can see on Figure 5.6 that the computation time is more important for all the instances in the *TSPGraphH* model than in the *TSPFF* model. This is due to the fact that, even if the number of nodes is smaller in the graph model, the time required to maintain, update and prune the graph structure is way bigger than the time required to try another assignment in the integer model. For example, for 25 cities, the integer model explores 20327 nodes per second while the graph model explores 420 nodes per second.

The fact the the number of nodes explored is lower in the graph model looks very promising but the actual implementation is not efficient enough.

TSP, TSPFF, TSPGraph and TSPGraphH

We can see on Figure 5.7 the evolution of the number of nodes explored in all the four models. This figure shows that the two models with heuristics *TSPFF* and *TSPGraphH* have better results than the two other models, as was expected.

We can see on Figure 5.8 the evolution of the computation time in all the four models. Here we can see that the time required is roughly the same for the basic integer model and the graph model with heuristic. This is due to the fact that, even if the number of nodes explored in the graph model with heuristic *TSPGraphH* is smaller than the number of nodes explored in the basic integer model, the computation needed to explore one node is bigger in the graph model.

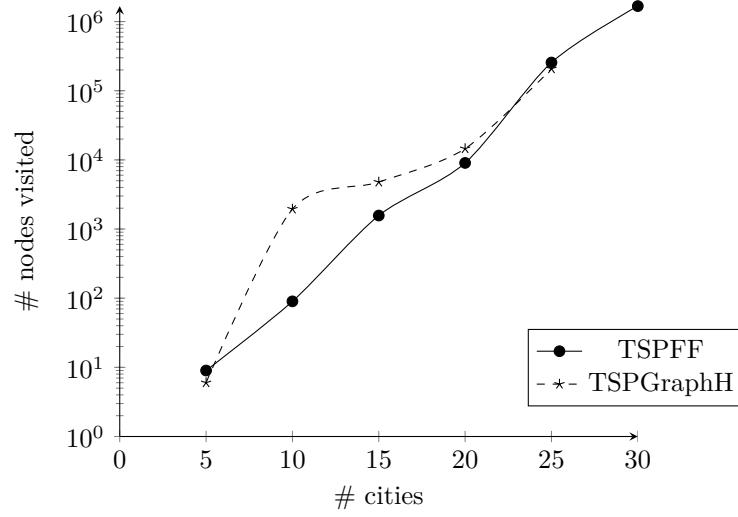


Figure 5.5: Illustration of the number of nodes visited in the models *TSPFF* and *TSPGraphH*

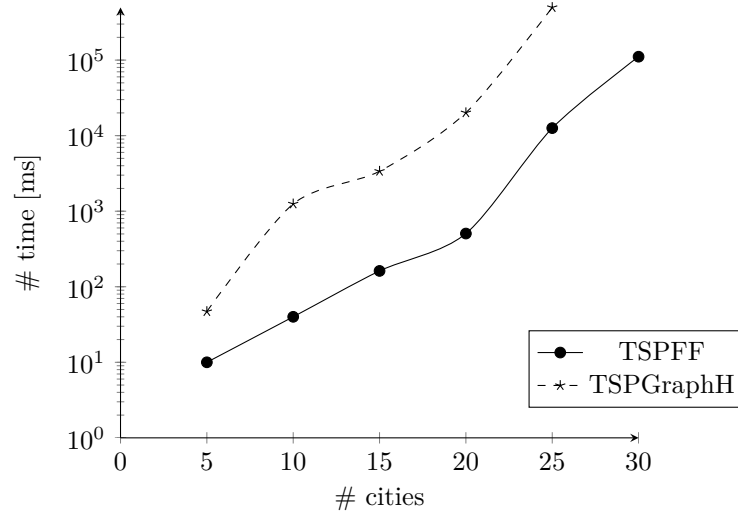


Figure 5.6: Illustration of the computation time in the models *TSP* and *TSP-Graph*

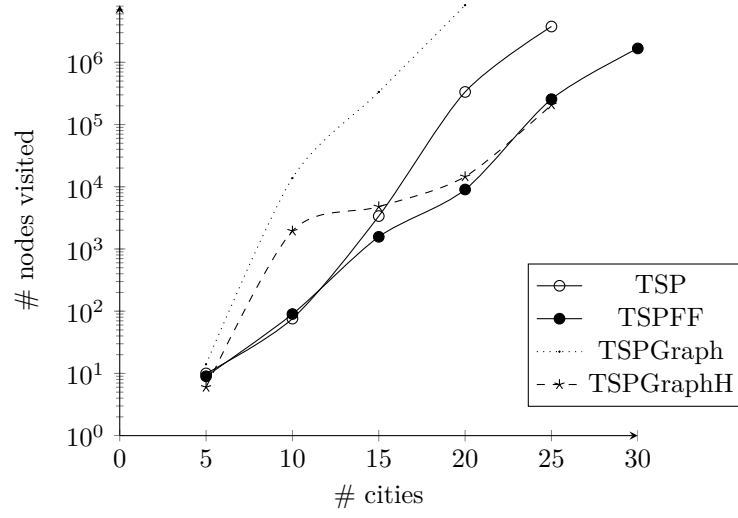


Figure 5.7: Illustration of the number of nodes visited in the four models

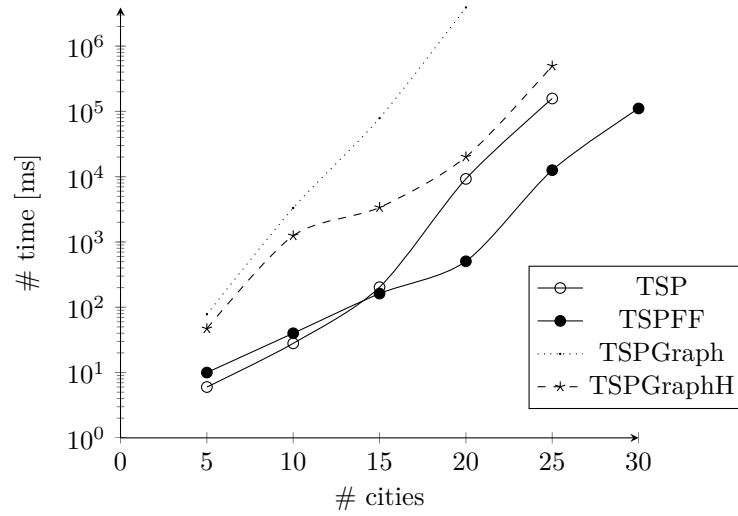


Figure 5.8: Illustration of the computation time in the four models

Chapter 6

Conclusion and future work

As presented in [1] and in [2], graph variables paradigm can be very useful in the constraint programming to model various applications. Some papers such as [3] indeed showed that using the graph structure can improve the resolution of the problem.

We then implemented in this work a graph variable which could be used in constraint programming. There are a lot of constraints that can be associated to this graph variable. We implemented here some of them. Additional constraints and their pruning rules can be found in [4].

The tests showed that using a graph structure allows to have a better search in the resolution of problems. The search is indeed guided by the graph structure and the constraint pruning rules allow to explore less nodes than a standard model without graphs would do. However, the current implementation is not efficient enough to be competitive with the integer models that are much better optimized.

We will now list some possible future works. A lot of constraints can be associated with graph variables, a continuation of this work could be to implement other constraints such as the spanning tree constraint presented in [4]. Another work could be to improve the current implementation of the graph variables and the associated constraints. Improvements can be made directly in the implementation or in the pruning rules.

Appendix A

TSP code examples

A.1 TSP instances and useful functions

The instances are available in the electronic version of this paper by clicking on the name of the instance: *tsp5 tsp10 tsp15 tsp20 tsp25 tsp30*

The class `Utils` that is used for the visualization and for loading the instances is electronically available *here*.

A.2 Integer Model

The integer model of the traveling salesman problem implemented by P. Schaus and R. Hartert is shown below.

```
object TSP extends CPModel with App {  
  // Data  
  val (distMatrix, coordinates) = TSPGenerator.loadInstance("tsp20.txt")  
  val nCities = distMatrix(0).length  
  val Cities = 0 until nCities  
  
  // Variables  
  val succ = Array.fill(nCities)(CPIntVar(Cities))  
  val totDist = CPIntVar(0 to distMatrix.flatten.sum)  
  
  // Constraints  
  add(circuit(succ), Strong)  
  add(minAssignment(succ, distMatrix, totDist))  
  add(sum(Cities)(i => distMatrix(i)(succ(i))) == totDist)  
  
  // Search heuristic  
  minimize(totDist) search binaryStatic(succ)  
  //minimize(totDist) search binaryFirstFail(succ)
```

```

// Visual Component
val visual = new VisualTSP(coordinates, succ)
var nSols = 0
onSolution {
  nSols += 1
  visual.updateTour(nSols, totDist.value)
}

val stats = start()
println(stats)
}

```

The complete class TSP is electronically available *here*. This class needs the solver Oscar [5] and the class Utils to be used on an instance.

A.3 Graph Model

Our implementation of the graph model of the traveling salesman problem is shown below.

```

object TSPGraph extends CPModel with App {
  // Data
  val (loadDistMatrix, loadCoordinates) =
    TSPGenerator.loadInstance("benchmarks/tsp/Instances/tsp10.txt")
  val nCities = loadDistMatrix(0).length
  val Cities = 0 until nCities

  // to model TSP with Graph, we will add a new node, lets call it 0'
  //   if there is n cities, they are noted from 0 to n-1; the city n
  //   will be the 0'

  // update distances and coordinates with 0'
  val distMatrix : Array[Array[Int]] = loadDistMatrix.map(a => a :+
    a(0)) :+ Array.fill(nCities+1)(0)
  val coordinates : Array[(Int,Int)] = loadCoordinates :+
    loadCoordinates(0)
  def dist(n1 : Int, n2 : Int) : Int = distMatrix(n1)(n2)

  // Variables
  val maxDist = distMatrix.flatten.sum
  val totDist = CPIntVar(0 to maxDist)
  val graph = CPGraphVar(nCities+1)
  graph.removeEdge(0, nCities)

  // Constraints
  // all nodes in the path are mandatory
  for (n <- 0 to nCities)
    add(graph.addNode(n))
}

```

```

// as we double starting point, we have to go from 0 to 0'
add(new GraphPath(graph,0,nCities,dist,totDist))

// Search heuristic
minimize(totDist) search {
  val requiredEdges : List[Int] =
    graph.requiredNodes.flatMap(graph.requiredEdges(_))
  val possibleNotRequiredEdges : List[Int] =
    graph.possibleNodes.flatMap(graph.possibleEdges(_)).filter(n
    => !requiredEdges.contains(n))

  if (possibleNotRequiredEdges.isEmpty){
    noAlternative
  }
  else {
    val e = possibleNotRequiredEdges.head
    val (s,d) = graph.edge(e)
    branch { add(graph.addEdge(s,d)) } { add(graph.removeEdge(s,d)) }
  }
}

// Visual Component
val visual = new VisualTSP_Graph(coordinates)
var nSols = 0
onSolution {
  nSols += 1
  var arr = Array.fill(nCities)(0)
  for (i <- 0 until nCities)
    arr(i) = graph.edge(graph.possibleOutEdges(i).head)._2
  visual.updateTour(nSols, totDist.value,arr)
}

val stats = start()
println(stats)
}

```

The entire TSPGraph class is electronically available *here*.
The entire TSPGraphH class is electronically available *here*.

Bibliography

- [1] G. Dooms, Y. Deville, and P. Dupont. CP(graph): Introducing a graph computation domain in constraint programming. 2005.
- [2] X. Lorca. *Contraintes de Partitionnement de Graphe*. PhD thesis, Université de Nantes, 2010.
- [3] J-G. Fages and X. Lorca. Improving the asymmetric tsp by considering graph structure. 2012.
- [4] G. Dooms. *The CP(Graph) Computation Domain in Constraint Programming*. PhD thesis, Université Catholique de Louvain, 2006.
- [5] Oscar Team. Oscar: Scala in OR, 2014. Available from <https://bitbucket.org/oscarlib/oscar>.
- [6] Andrew Lambert. Fork of oscar: implementation of graph variable and associated constraints, 2014. Available from <https://bitbucket.org/andlambert/oscar/src>.
- [7] G. Dooms and Z. Zampelli. Homepage of cp(graph), cp(map) and related software, 2005. Available from <http://cpgraph.info.ucl.ac.be/>.
- [8] Choco Team. choco3: An open-source java library for constraint programming, 2014. Available from <https://github.com/chocoteam/choco3>.
- [9] Choco Team. choco3 wiki: Variables, 2014. Available from <https://github.com/chocoteam/choco3/wiki/Variables>.
- [10] V. le Clément, P. Schaus, C. Solnon, and C. Lecoutre. Sparse-sets for domain implementation. 2013.
- [11] K. Apt. *Principles of constraint programming*. Cambridge University Press, 2003.
- [12] J.A. Bondy and U.S.R Murty. *Graph Theory*. Springer, 2008.
- [13] W.D. Wallis. *A beginner's guide to Graph Theory*. Springer, 2010.
- [14] P. Schaus. Cp for the impatient, 2014. Available from <http://www.inf.ucl.ac.be/~pschaus/cp4impatient/index.html>.

- [15] C. Gervet. Interval propagation to reason about sets: Definition and implementation of a practical language. *In constraints*, pages 191–244, 1997.
- [16] G. Doms and I. Katriel. Graph constraints in constraint programming: Weighted spanning trees. 2005.
- [17] M. Sellmann. Cost-based filtering for shorter path constraints. 2005.
- [18] M. Sellmann. Reduction techniques in constraint programming and combinatorial optimization. 2003.
- [19] G. Brassard and P. Bratley. *Fundamentals of algorithmics*. Prentice Hall, 1996.
- [20] Y. Caseau and F. Laburthe. Solving small tsps with constraints. 1997.
- [21] F. Focacci, A. Lodi, and M. Milano. Embedding relaxations in global constraints for solving tsp and tsptw. 2002.